Multithreading and Concurrency

#	Scenario (Short)	Topic / Concept	Level	Role Focus
1	Thread lifecycle in action	Thread Basics & Lifecycle	L1	Backend Core
2	Runnable vs Thread in a web application	Thread Basics & Lifecycle	L1	Backend Core
3	Synchronization for shared resource	Thread Basics & Lifecycle	L2	Backend Core
4	Producer–consumer using wait/notify	Thread Communication & Locks	L2	Backend Core
5	Deadlock prevention in resource allocation	Thread Communication & Locks	L3	Backend Core
6	ReentrantLock vs synchronized in real apps	Thread Communication & Locks	L3	Advanced Backend
7	Fair lock for printing queue	Thread Communication & Locks	L3	Advanced Backend
8	ExecutorService for background jobs	Executors & Thread Pooling	L2	Backend Core
9	Callable and Future for report generation	Executors & Thread Pooling	L3	Backend Core
10	Safe ExecutorService shutdown	Executors & Thread Pooling	L3	Backend Core
11	ScheduledExecutorService for reminders	Executors & Thread Pooling	L2	Backend Core
12	CountDownLatch for task synchronization	Advanced Concurrency Utilities	L3	Backend Core
13	CyclicBarrier for multi-threaded simulation	Advanced Concurrency Utilities	L3	Backend Core
14	Semaphore for rate limiting	Advanced Concurrency Utilities	L3	Backend Core
15	BlockingQueue for thread-safe order processing	Advanced Concurrency Utilities	L3	Backend Core

16	Parallel Stream for bulk computation	Parallel Streams & CompletableFuture	L2	Backend Core
17	CompletableFuture for async API calls	Parallel Streams & CompletableFuture	L3	Backend Core
18	Error handling in CompletableFuture	Parallel Streams & CompletableFuture	L3	Backend Core
19	Chaining CompletableFutures	Parallel Streams & CompletableFuture	L3	Backend Core
20	AtomicInteger for thread-safe counters	Thread Safety & Data Consistency	L2	Backend Core
21	ConcurrentHashMap for caching	Thread Safety & Data Consistency	L3	Backend Core
22	Volatile keyword for visibility	Thread Safety & Data Consistency	L3	Backend Core
23	Double-checked locking with volatile	Thread Safety & Data Consistency	L3	Backend Core
24	ThreadLocal for request-scoped data	Thread Safety & Data Consistency	L3	Backend Core
25	Producer-consumer: BlockingQueue vs wait-notify	Thread Safety & Data Consistency	L3	Backend Core
26	ExecutorService tuning for large workloads	Executors & Optimization	L3	Advanced Backend
27	Asynchronous logging using queue	Thread Safety & Data Consistency	L3	Backend Core
28	Real-time leaderboard updates	Thread Safety & Data Consistency	L3	Full Stack
29	Using ReadWriteLock for shared data	Thread Safety & Data Consistency	L3	Backend Core
30	Two-way money transfer (avoid deadlock)	Deadlock & Locks	L3	Advanced Backend
31	Mixed-duration jobs with limited workers	Thread Pooling	L3	Advanced Backend

32	Cap concurrent client connections	Semaphore / Concurrency Control	L3	Backend Core
33	Start many tasks → handle results as ready	ExecutorCompletionService	L3	Backend Core
34	Producer hands off data; consumer waits until ready	BlockingQueue / Handoff	L3	Backend Core
35	Threads grabbing different locks (deadlock risk)	Deadlock Analysis	L3	Advanced Backend
36	Many quick tasks — avoid thread per task	Thread Pooling Strategy	L3	Advanced Backend
37	Wait for all subsystems before startup	CountDownLatch	L2	Backend Core
38	Multi-phase workers rendezvous each step	CyclicBarrier	L3	Backend Core
39	Speed up independent list processing	Parallel Streams	L2	Backend Core
40	Ensure JVM waits for critical work before exit	Thread Join / Daemon	L2	Backend Core
41	Extend base class + run concurrently (Runnable composition)	Runnable vs Inheritance	L2	Backend Core
42	Prevent inconsistent balance via synchronization	Synchronization	L2	Backend Core
43	Consumer waits forever (fix missed notify)	Thread Communication Bug	L3	Backend Core
44	Avoid deadlock in two-account transfer	Deadlock Prevention	L3	Backend Core
45	Prevent starvation under heavy contention	Fair Lock / ReentrantLock	L3	Backend Core
46	Run recurring tasks reliably	ScheduledExecutorService	L2	Backend Core
47	Fan-out to multiple services and aggregate safely	CompletableFuture AllOf	L3	Backend Core

48	Producer-consumer with backpressure & multiple consumers	BlockingQueue / Flow Control	L4	Advanced Backend
49	Dynamic team size across phases (CyclicBarrier limitation)	Phaser Utility	L4	Advanced Backend
50	Time-box and cancel async pipeline	CompletableFuture Timeout	L4	Advanced Backend
51	Parallel stream corrupts shared state	Parallel Stream Safety	L3	Backend Core
52	Threads must finish init before main continues	CountDownLatch / Join	L2	Backend Core
53	Parallel + Sequential phases in pipeline	ForkJoin / Custom Executor	L4	Advanced Backend
54	Cancel long-running tasks past deadline	Future.cancel() / Timeout	L3	Backend Core
55	Parallel stream slower than sequential (small data)	Performance Tuning	L2	Backend Core
56	Multiple readers, single writer pattern	ReadWriteLock	L3	Backend Core
57	Coordinating ordered tasks	Chained Futures	L3	Backend Core
58	Detect and recover from livelock	Livelock Detection	L4	Advanced Backend
59	Cancel tasks cleanly in ExecutorService	Graceful Shutdown	L3	Backend Core
60	Parallel aggregation of large datasets	ForkJoinPool	L3	Advanced Backend
61	Throttle API calls with limited concurrency	Semaphore / Rate Limiter	L3	Backend Core
62	Coordinate variable participants across phases	Phaser	L4	Advanced Backend
63	Handle task dependencies with CompletableFuture	CompletableFuture Dependency	L3	Backend Core

64	Handle starvation in lock-heavy system	Lock Tuning / Fairness	L4	Advanced Backend
65	Run thousands of tasks efficiently	ThreadPoolExecutor Tuning	L4	Advanced Backend
66	Divide-and-conquer with ForkJoinPool	ForkJoin Framework	L4	Advanced Backend
67	Process results as soon as they finish	ExecutorCompletionService	L3	Backend Core
68	Robust async pipeline with fallbacks	CompletableFuture + Handle	L4	Advanced Backend
69	Double-buffer handoff between two threads	Exchanger Utility	L4	Advanced Backend
70	Graceful shutdown of multi-consumer pipeline (poison pill)	BlockingQueue Pattern	L4	Advanced Backend

Scenario 1: Thread lifecycle in action

Imagine you are building a logging and notification system where multiple worker threads are created. Some threads write logs, others send notifications. To debug and optimize performance, you want to track the exact states each thread goes through during its lifetime.

Answer:

A thread in Java doesn't just jump from start to end; it goes through multiple well-defined states: New, Runnable, Blocked, Waiting, Timed Waiting, and Terminated.

- When I create a thread object, it is in New.
- Once I call start(), it moves to Runnable — ready to run, but not guaranteed to be running immediately.
- If it tries to acquire a lock held by another thread, it becomes Blocked.
- If it's waiting indefinitely for a signal (like Object.wait()), it's in Waiting.
- If it sleeps or waits with a timeout, it's in Timed Waiting.
- Once run() finishes, it goes to Terminated.

Code:

```
class ThreadLifecycleDemo extends Thread {
  public void run() {
    System.out.println("Thread is running...");
    try {
      Thread.sleep(200); // Timed Waiting
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }

  public static void main(String[] args) throws InterruptedException {
    ThreadLifecycleDemo t = new ThreadLifecycleDemo();
    System.out.println("Initial: " + t.getState()); // NEW
    t.start();
    System.out.println("After start: " + t.getState()); // RUNNABLE
    Thread.sleep(50);
    System.out.println("During execution: " + t.getState()); // RUNNABLE or
TIMED_WAITING
    t.join();
    System.out.println("Final: " + t.getState()); // TERMINATED
  }
}
```

Reasoning to interviewer:

- getState() gives only a snapshot; transitions are fast, so states may vary between runs.
- Runnable means eligible for execution, not guaranteed to be actively running.
- Blocked is specific to lock contention, while Waiting/Timed Waiting are about coordination or sleeping.

Future scope:

- Use JMX, thread dumps, or monitoring tools in production for accurate state tracking.
- Naming threads (like “log-writer-1”) makes debugging easier.
- For large apps, manage lifecycles via ExecutorService instead of raw threads.

Scenario 2: Runnable vs Thread in a web application

Suppose you are working on a web application where every incoming request spawns a background job, like recalculating discounts. You need to decide whether to represent that job by extending Thread or by implementing Runnable.

Answer:

I prefer implementing Runnable because it keeps the task separate from the execution mechanism. Runnable lets me reuse the same task logic across different environments — a single thread, a thread pool, or a scheduler. If I extend Thread, I couple my business logic with threading behavior, and I also lose the ability to extend another class.

Code:

```
class DiscountTask implements Runnable {
   private final String orderId;

   DiscountTask(String orderId) {
      this.orderId = orderId;
   }

   @Override
   public void run() {
      System.out.println("Recalculating discounts for order " + orderId);
   }
}

public class RunnableVsThreadDemo {
   public static void main(String[] args) {
      Runnable task = new DiscountTask("ORD123");
      Thread t = new Thread(task); // Pass Runnable into Thread
      t.start();
   }
}
```

Reasoning to interviewer:

- Runnable focuses on “what to do,” Thread focuses on “how to run.”
- With Runnable, I can pass the same task to an ExecutorService for pooled execution.
- Extending Thread ties the logic and execution together, which is rarely ideal in production.

Future scope:

- For returning values, switch to Callable with Future.
- Prefer ExecutorService or ScheduledExecutorService to manage threads at scale.
- With Loom (virtual threads), Runnable becomes even more important for lightweight concurrency.

Scenario 3: Synchronization for shared resource

Consider an online ticket booking system where multiple users try to book tickets at the same time. If two threads update the available seat count simultaneously without proper synchronization, overbooking can occur.

Answer:

This is a classic race condition. To fix it, I ensure only one thread modifies the shared resource at a time. Java provides synchronized methods/blocks, explicit locks like ReentrantLock, or lock-free atomic classes for counters.

Code:

```
class TicketCounter {
   private int seats = 1;

   public synchronized boolean bookSeat(String user) {
      if (seats > 0) {
         seats--;
         System.out.println(user + " successfully booked a seat.");
         return true;
      }
      System.out.println(user + " failed to book. No seats left.");
      return false;
   }
}

public class BookingDemo {
   public static void main(String[] args) {
      TicketCounter counter = new TicketCounter();

      Thread t1 = new Thread(() -> counter.bookSeat("User-1"));
      Thread t2 = new Thread(() -> counter.bookSeat("User-2"));

      t1.start();
      t2.start();
   }
}
```

Reasoning to interviewer:

- synchronized ensures only one thread enters bookSeat() at a time.
- This prevents two users from booking the last seat simultaneously.
- For advanced cases, ReentrantLock provides timed/interruptible locks.

- For counters like page views, AtomicInteger is lighter than synchronization.

Future scope:

- Use ReadWriteLock when reads dominate over writes.
- For distributed systems, combine Java synchronization with database-level locking or optimistic concurrency.
- Monitor contention — sometimes redesigning data structures avoids locks altogether.

Scenario 4: Aggregate multiple API calls asynchronously

You're building an e-commerce dashboard that pulls product info, pricing, and inventory from three different services. You want all three requests to run in parallel and then render the page only after every piece is ready. If any one fails, you still want clear error handling and possibly a fallback.

Answer:

I use CompletableFuture to fire off independent calls concurrently (supplyAsync), compose dependent calls (thenCompose), and merge results (thenCombine/allOf). I prefer a custom Executor so these tasks don't fight the common fork-join pool used elsewhere. I also attach exception handling (exceptionally/handle) and timeouts. For final aggregation, allOf works well; for pairwise composition, thenCombine is cleaner.

Code:

```
import java.util.concurrent.*;
import java.time.Duration;
import java.util.Map;

public class AsyncAggregate {
   private static final ExecutorService ioPool =
         Executors.newFixedThreadPool(10, r -> {
            Thread t = new Thread(r, "io-pool-" + System.nanoTime());
            t.setDaemon(true);
            return t;
         });

   // Mock I/O calls
   static String fetchProduct(String id)   { sleep(200); return "Product:" + id; }
   static String fetchPrice(String id)     { sleep(150); return "Price:₹999"; }
   static String fetchInventory(String id) { sleep(180); return "Inventory:12"; }

   // Simple timeout helper
   static <T> CompletableFuture<T> withTimeout(CompletableFuture<T> cf, Duration
d) {
      CompletableFuture<T> timeout = new CompletableFuture<>();
      ScheduledExecutorService ses =
Executors.newSingleThreadScheduledExecutor();
      ses.schedule(() -> timeout.completeExceptionally(new TimeoutException()),
d.toMillis(), TimeUnit.MILLISECONDS);
      return cf.applyToEither(timeout, t -> t).whenComplete((r, e) -> ses.shutdown());
   }

   public static void main(String[] args) {
      String productId = "SKU-42";
```

```
    CompletableFuture<String> fProduct   =
      CompletableFuture.supplyAsync(() -> fetchProduct(productId), ioPool);

    CompletableFuture<String> fPrice     =
      CompletableFuture.supplyAsync(() -> fetchPrice(productId), ioPool);

    CompletableFuture<String> fInventory =
      CompletableFuture.supplyAsync(() -> fetchInventory(productId), ioPool);

    // Combine all three
    CompletableFuture<Map<String, String>> combined =
      CompletableFuture.allOf(fProduct, fPrice, fInventory)
        .thenApply(v -> Map.of(
          "product",   fProduct.join(),   // safe here: allOf has completed
          "price",     fPrice.join(),
          "inventory", fInventory.join()
        ))
        .exceptionally(ex -> {
          // central fallback or partial data strategy
          System.err.println("Aggregation failed: " + ex);
          return Map.of("error", "partial/unavailable");
        });

    // Add timeout and final consumption
    Map<String, String> result =
      withTimeout(combined, Duration.ofSeconds(1))
        .handle((ok, ex) -> ex == null ? ok : Map.of("error", "timeout"))
        .join(); // join: unchecked CompletionException on failure

    System.out.println(result);
    ioPool.shutdown();
  }

  static void sleep(long ms) { try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } }
}
```

Reasoning to interviewer:

- supplyAsync runs each I/O task on a dedicated pool so CPU-bound work isn't starved.
- allOf is convenient when I just need "wait for all"; I then safely read each result with join because all futures are completed by that point. For two-way merges,

thenCombine keeps types explicit and avoids joins.

• I attach error handlers at the right granularity: locally (each future with exceptionally) or globally (the combined stage).

• I prefer join in pipelines (unchecked) and get when I need checked exceptions or timeouts directly on a future.

• I avoid blocking calls inside async stages unless I'm on an I/O pool; blocking inside the common pool risks thread starvation.

Future scope:

• Use thenCompose for dependent calls (e.g., fetch product → then fetch recommendations).

• Introduce circuit breakers/bulkheads (Resilience4j) for flaky upstreams.

• Switch to virtual threads for simpler blocking I/O without large pools, while keeping the same CompletableFuture composition style.

Scenario 5: Correctly updating a shared counter under contention

In a rate-limiter for an API, multiple request-handler threads increment a per-minute counter. Under load, you see inconsistent counts and occasional negative values due to interleaving updates.

Answer:

I pick the lightest correct tool for the contention pattern. For simple atomic increments, AtomicInteger (or LongAdder under high contention) gives lock-free performance. If the update is more than a single atomic op (read-modify-write with checks), I use synchronized or ReentrantLock and keep the critical section tiny.

Code (AtomicInteger for simple counters):

```
import java.util.concurrent.atomic.AtomicInteger;

class RequestCounter {
   private final AtomicInteger count = new AtomicInteger();

   public int incrementAndGet() {
      return count.incrementAndGet();
   }
   public int snapshot() { return count.get(); }
}
```

Code (LongAdder for very high contention hot spots):

```
import java.util.concurrent.atomic.LongAdder;

class HotCounter {
   private final LongAdder adder = new LongAdder();
   public void increment() { adder.increment(); }
   public long sum() { return adder.sum(); }
}
```

Code (when the update has rules, e.g., don't exceed a cap):

```
import java.util.concurrent.locks.ReentrantLock;

class CappedCounter {
   private final ReentrantLock lock = new ReentrantLock();
   private int value = 0;
   private final int cap;
```

```
  CappedCounter(int cap) { this.cap = cap; }

  public boolean tryIncrement() {
    lock.lock();
    try {
      if (value >= cap) return false;
      value++;
      return true;
    } finally {
      lock.unlock();
    }
  }

  public int snapshot() {
    lock.lock();
    try { return value; }
    finally { lock.unlock(); }
  }
}
```

Reasoning to interviewer:

- AtomicInteger is perfect for single-variable atomic ops; no blocking, good scalability.
- LongAdder spreads contention across cells and is superior under high write concurrency, with slightly stale reads during summation—which is fine for metrics.
- When updates need conditional logic (caps, invariants), I switch to explicit locking and minimize the locked region. I never expose the lock object; I keep it private to avoid external interference.
- I avoid naïve "read, then write" without atomicity because it invites lost updates. If I must do CAS loops (update based on current value), I use compareAndSet in a small loop.

Future scope:

- If reads dominate and writes are rare, consider ReadWriteLock/StampedLock.
- For distributed counters, add DB/Redis layer with atomic increments or optimistic concurrency (version columns).

• Expose metrics (contention rate, failed increments) to catch hotspots and decide between LongAdder vs locks vs sharding the counter.

Scenario 6: Exclusive DB update without collisions

Two threads attempt to update the same customer row—say, applying a wallet credit and recording a refund. Only one should enter the critical section at a time so the balance doesn't flip-flop.

Answer:

I'll guard the critical section with a ReentrantLock. The thread that acquires the lock executes the DB operation; the other waits (or times out) and tries later. I always unlock in a finally block so exceptions don't leave the lock held.

Code:

```
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class CustomerWalletService {
  private final Lock dbLock = new ReentrantLock(); // per-customer or per-resource
lock

  public void applyCredit(String customerId, int amount) {
    boolean acquired = false;
    try {
      // Avoid indefinite blocking; back off if we can't get the lock in time
      acquired = dbLock.tryLock(200, TimeUnit.MILLISECONDS);
      if (!acquired) {
        throw new IllegalStateException("Busy updating wallet, try again");
      }

      // Critical section: read-modify-write must be atomic
      int current = readBalance(customerId);
      int updated = current + amount;
      writeBalance(customerId, updated);
      audit(customerId, "CREDIT", amount, updated);

    } catch (InterruptedException ie) {
      Thread.currentThread().interrupt(); // preserve interrupt status
      throw new RuntimeException("Interrupted while waiting for lock", ie);
    } finally {
      if (acquired) dbLock.unlock();
    }
  }

  // Mock I/O
  private int readBalance(String id) { /* SELECT ... FOR UPDATE */ return 100; }
```

```
    private void writeBalance(String id, int val) { /* UPDATE ... */ }
    private void audit(String id, String type, int delta, int newBal) { /* INSERT ... */ }
}

public class LockDemo {
    public static void main(String[] args) {
        CustomerWalletService svc = new CustomerWalletService();
        Thread t1 = new Thread(() -> svc.applyCredit("C42", 50), "credit-1");
        Thread t2 = new Thread(() -> svc.applyCredit("C42", 30), "credit-2");
        t1.start(); t2.start();
    }
}
```

Reasoning to interviewer:

- ReentrantLock gives me tryLock with timeout and interruptible acquisition—clean for service shutdowns and avoiding deadlocks.
- The critical section contains only the read-modify-write and audit; validation and retries live outside to reduce lock hold time.
- I keep the lock private so no external code can accidentally create lock-ordering cycles.

Future scope:

- If multiple rows can be touched, establish a global ordering for locks to avoid deadlocks.
- For DB consistency across crashes, pair this with database-level row locks or optimistic concurrency (version columns).
- If throughput is hurt by one big lock, shard locks by customerId (e.g., striped locks).

Scenario 7: Producer makes parts, assembler must wait

In a car plant sim, one thread produces a part; another assembles the car. The assembler must block until a part is available, and the producer must block if the previous part wasn't consumed.

Answer:

I'll use the intrinsic monitor with wait/notifyAll. The rule is "check in a loop" to handle spurious wakeups, and use notifyAll when there could be multiple waiters. For production systems I'd prefer higher-level constructs (BlockingQueue or Conditions), but wait/notify is fine to demonstrate fundamentals.

Code:

```
class Factory {
   private boolean partAvailable = false;

   public synchronized void produce() throws InterruptedException {
      while (partAvailable) {          // producer waits until part is consumed
         wait();
      }
      partAvailable = true;
      System.out.println("Part produced");
      notifyAll();                     // wake assembler(s)
   }

   public synchronized void assemble() throws InterruptedException {
      while (!partAvailable) {         // assembler waits until part is made
         wait();
      }
      partAvailable = false;
      System.out.println("Car assembled");
      notifyAll();                     // let producer proceed
   }
}

public class WaitNotifyDemo {
   public static void main(String[] args) {
      Factory f = new Factory();
      Thread producer = new Thread(() -> {
         try { for (int i=0;i<3;i++) f.produce(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
      }, "producer");

      Thread assembler = new Thread(() -> {
```

```
        try { for (int i=0;i<3;i++) f.assemble(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
    }, "assembler");

    producer.start();
    assembler.start();
  }
}
```

Reasoning to interviewer:

- wait and notifyAll must be called while holding the same monitor (synchronized on the same object), otherwise IllegalMonitorStateException.
- I use while, not if, to re-check the condition after waking—protects against spurious wakeups and missed signals.
- notifyAll avoids missed wakeups when there are multiple producers/assemblers; narrow to notify only when you can prove one waiter and no lost-notify risks.

Future scope:

- Replace wait/notify with a BlockingQueue—it's simpler and safer for multi-producer/multi-consumer.
- Or use ReentrantLock + Condition for explicit condition queues (await/signal).
- Add timeouts to avoid permanent stalls and emit metrics on queue depth/latency.

Scenario 8: Gracefully shutting down an ExecutorService

You've submitted a batch of background tasks (email sends, invoice generation). When the request completes or the service is stopping, you need to stop taking new work, let in-flight tasks finish, and not leak threads.

Answer:

The sequence is shutdown → awaitTermination → (optional) shutdownNow. I also propagate interrupts so tasks can stop promptly, and I add a final await after shutdownNow to give tasks a chance to react.

Code:

```
import java.util.List;
import java.util.concurrent.*;

public class ExecutorShutdownDemo {
  public static void main(String[] args) {
    ExecutorService pool = Executors.newFixedThreadPool(3);

    for (int i = 0; i < 5; i++) {
      pool.submit(() -> {
        try {
          System.out.println(Thread.currentThread().getName() + " working");
          Thread.sleep(300); // simulate work
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt(); // respond to shutdownNow
        }
      });
    }

    // graceful shutdown
    pool.shutdown(); // stop accepting new tasks

    try {
      if (!pool.awaitTermination(2, TimeUnit.SECONDS)) {
        List<Runnable> dropped = pool.shutdownNow(); // cancel queued tasks,
interrupt running
        System.out.println("Cancelled queued: " + dropped.size());
        if (!pool.awaitTermination(2, TimeUnit.SECONDS)) {
          System.err.println("Pool did not terminate cleanly");
        }
      } else {
        System.out.println("All tasks finished normally");
      }
    } catch (InterruptedException ie) {
```

```
            pool.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}
```

Reasoning to interviewer:

- shutdown is idempotent: it rejects new tasks but lets running ones finish. awaitTermination gives them time.
- If time's up, shutdownNow interrupts running tasks and drains the queue; well-written tasks check interruption and exit quickly.
- I always handle InterruptedException by restoring interrupt status—this matters if a higher layer wants to observe it (e.g., server shutdown).
- Returning and logging the dropped queue helps visibility into what didn't run.

Future scope:

- Wrap the ExecutorService in a lifecycle manager and add a JVM shutdown hook for safety.
- Prefer try-with-resources with ExecutorService in newer Java (using an AutoCloseable wrapper) or use structured concurrency when available.
- Replace ad-hoc threads with CompletableFuture or virtual threads where blocking is simpler and shutdowns are lighter.

Scenario 8: Process results as soon as tasks finish

In a stock-trading app, you fire off multiple API calls in parallel: one for stock prices, one for forex rates, and one for crypto values. Each service responds at different speeds. Instead of waiting for all results at the end, you want to process each result immediately as soon as it finishes.

Answer:

I use ExecutorCompletionService. It wraps an ExecutorService and provides a queue of completed tasks. Unlike a plain Executor, I can call take() or poll() to get results in the order they finish, not the order I submitted them.

Code:

```
import java.util.concurrent.*;

class MarketDataTask implements Callable<String> {
   private final String source;
   private final int delay;

   MarketDataTask(String source, int delay) {
      this.source = source;
      this.delay = delay;
   }

   @Override
   public String call() throws InterruptedException {
      Thread.sleep(delay);
      return source + " data ready";
   }
}

public class CompletionServiceDemo {
   public static void main(String[] args) throws Exception {
      ExecutorService pool = Executors.newFixedThreadPool(3);
      ExecutorCompletionService<String> ecs = new
ExecutorCompletionService<>(pool);

      ecs.submit(new MarketDataTask("Stocks", 1200));
      ecs.submit(new MarketDataTask("Forex", 500));
      ecs.submit(new MarketDataTask("Crypto", 800));

      for (int i = 0; i < 3; i++) {
         Future<String> f = ecs.take(); // waits for whichever finishes next
         System.out.println(f.get());
      }
```

```
    pool.shutdown();
  }
}
```

Reasoning to interviewer:

- A plain ExecutorService only gives Futures in submission order.
- ExecutorCompletionService internally uses a BlockingQueue to queue finished tasks, so I can consume them in completion order.
- This way, if “Forex” finishes before “Stocks,” I can process it immediately instead of waiting.

Future scope:

- Use poll(timeout) instead of take() to avoid blocking indefinitely.
- Add exception handling with Future.isCancelled() or try/catch on get().
- For streaming pipelines, couple CompletionService with backpressure and batching.

Scenario 9: Dependent asynchronous tasks

In an order system, Task A validates payment. Only if payment succeeds should Task B dispatch the order. Task B should never start until Task A has finished successfully.

Answer:

CompletableFuture makes this straightforward with thenCompose. It waits for Task A to complete, then starts Task B, passing along the result. This avoids blocking and keeps everything asynchronous.

Code:

```
import java.util.concurrent.*;

public class DependentTasksDemo {
   public static void main(String[] args) throws Exception {
      ExecutorService pool = Executors.newFixedThreadPool(2);

      CompletableFuture<String> payment = CompletableFuture.supplyAsync(() -> {
         try { Thread.sleep(800); } catch (InterruptedException ignored) {}
         return "Payment confirmed";
      }, pool);

      CompletableFuture<String> dispatch = payment.thenCompose(paymentResult -
>
         CompletableFuture.supplyAsync(() -> {
            System.out.println("Starting dispatch after: " + paymentResult);
            try { Thread.sleep(800); } catch (InterruptedException ignored) {}
            return "Order dispatched";
         }, pool)
      );

      System.out.println(dispatch.get());
      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- thenCompose is for dependent workflows: Task B won't even start until Task A's result is available.
- thenCombine would be for two independent tasks whose results I merge.
- This avoids writing nested Future.get() calls, which block threads unnecessarily.

Future scope:

- Chain multiple steps: payment → invoice → shipping → notification.
- Add exceptionally() to handle failures in Task A and skip Task B.
- Use allOf() or anyOf() when coordinating multiple independent futures.

Scenario 10: Retrying flaky API calls with timeout

You're calling an external shipping API that sometimes hangs or fails. You want to retry up to three times, with a 2-second timeout for each attempt, stopping as soon as one call succeeds.

Answer:

I wrap the API call in a CompletableFuture and apply get(timeout). If it times out or throws, I retry until the max attempts is reached. I always restore interrupts and shut down the executor at the end.

Code:

```
import java.util.concurrent.*;

class ShippingAPI implements Callable<String> {
   private static int attempt = 0;

   @Override
   public String call() throws Exception {
      attempt++;
      if (attempt < 3) {
         System.out.println("Attempt " + attempt + " failed.");
         throw new TimeoutException("Simulated failure");
      }
      return "API success on attempt " + attempt;
   }
}

public class RetryWithTimeoutDemo {
   public static void main(String[] args) throws Exception {
      ExecutorService pool = Executors.newSingleThreadExecutor();
      int maxRetries = 3;
      String response = null;

      for (int i = 1; i <= maxRetries; i++) {
         CompletableFuture<String> future =
            CompletableFuture.supplyAsync(() -> {
               try { return new ShippingAPI().call(); }
               catch (Exception e) { throw new CompletionException(e); }
            }, pool);

         try {
            response = future.get(2, TimeUnit.SECONDS); // timeout
            break; // success, no more retries
         } catch (TimeoutException | ExecutionException e) {
```

```
            System.out.println("Attempt " + i + " timed out/failed");
        }
      }

      if (response == null) {
        System.out.println("All attempts failed");
      } else {
        System.out.println(response);
      }

      pool.shutdown();
    }
}
```

Reasoning to interviewer:

- CompletableFuture.supplyAsync launches each attempt without blocking the main thread.
- get(timeout) enforces per-attempt time limits.
- Wrapping call() in CompletionException lets me propagate checked exceptions inside the async pipeline.
- Retry stops as soon as a success is returned, avoiding wasted attempts.

Future scope:

- Add exponential backoff between retries to avoid hammering the API.
- Switch to ScheduledExecutorService for controlled retry delays.
- Use a resilience library like Resilience4j for standardized retries, fallbacks, and circuit breakers.

Scenario 11: Phase-by-phase coordination across threads

You're running a multi-stage analytics job (extract → transform → load). Three worker threads perform work for "Phase 1," and no one should start "Phase 2" until all three have finished Phase 1. This repeats for several phases.

Answer:

I'll use a CyclicBarrier with the number of parties equal to the worker count. Each thread does its phase work, calls barrier.await() to wait for the others, and only then proceeds. Because the barrier is cyclic, it automatically resets for the next phase. I can also attach a barrier action to run exactly once per phase (e.g., log "phase done", rotate temp files).

Code:

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

class PhaseWorker implements Runnable {
   private final CyclicBarrier barrier;
   private final int totalPhases;
   private final int id;

   PhaseWorker(int id, CyclicBarrier barrier, int totalPhases) {
      this.id = id;
      this.barrier = barrier;
      this.totalPhases = totalPhases;
   }

   @Override
   public void run() {
      try {
         for (int phase = 1; phase <= totalPhases; phase++) {
            // Do phase-specific work
            System.out.printf("T%d: working in phase %d%n", id, phase);
            // Synchronize with peers at the barrier
            barrier.await();
            // Now all peers have completed this phase
            System.out.printf("T%d: proceed to next after phase %d%n", id, phase);
         }
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } catch (BrokenBarrierException e) {
         System.out.printf("T%d: barrier broken, aborting%n", id);
      }
   }
```

```
}

public class CyclicBarrierDemo {
   public static void main(String[] args) {
      int workers = 3;
      int phases = 3;

      CyclicBarrier barrier = new CyclicBarrier(
         workers,
         () -> System.out.println("-- all threads reached barrier; phase complete --")
      );

      for (int i = 1; i <= workers; i++) {
         new Thread(new PhaseWorker(i, barrier, phases), "worker-" + i).start();
      }
   }
}
```

Reasoning to interviewer:

- With a fixed set of worker threads looping over phases, CyclicBarrier cleanly gates progress per phase and automatically resets.
- The barrier action runs once when the last party arrives—useful for per-phase consolidation or logging.
- If any thread is interrupted or fails, the barrier breaks and others see BrokenBarrierException, which is safer than silently proceeding.

Future scope:

- If I also need to stop early when any thread fails validation, I can call barrier.reset() or propagate interrupts to break the phase.
- For dynamic party counts, consider Phaser, which supports registration/deregistration.
- For one-off rendezvous (single phase), CountDownLatch is lighter.

Scenario 12: Enforce per-task time limits with clean cancellation

In an image-processing pipeline, each filter must complete within 2 seconds. If a filter hangs or throws, it should be interrupted and the error recorded, then the pipeline should move on.

Answer:

I'll submit the task to an ExecutorService and use Future.get(timeout, unit). On timeout I call future.cancel(true) to interrupt the running thread. Inside the task, I write it to respond to interruption (propagate or clean up). I also unwrap ExecutionException to handle root causes.

Code:

```
import java.util.concurrent.*;

class FilterTask implements Callable<String> {
  @Override
  public String call() throws Exception {
    try {
      // Simulate work that might block
      for (int i = 0; i < 10; i++) {
        if (Thread.currentThread().isInterrupted()) {
          throw new InterruptedException("Filter interrupted");
        }
        Thread.sleep(300); // chunked sleeps to be interruption-friendly
      }
      return "OK";
    } catch (InterruptedException ie) {
      // Cleanup if needed, then rethrow
      throw ie;
    }
  }
}

public class TimeoutAndErrorHandlingDemo {
  public static void main(String[] args) {
    ExecutorService executor = Executors.newSingleThreadExecutor();
    Future<String> future = executor.submit(new FilterTask());
    try {
      String result = future.get(2, TimeUnit.SECONDS);
      System.out.println("Result: " + result);
    } catch (TimeoutException te) {
      System.out.println("Timed out; cancelling...");
      future.cancel(true); // interrupt running task
    } catch (ExecutionException ee) {
```

```
            Throwable cause = ee.getCause();
            System.out.println("Task failed: " + cause);
        } catch (InterruptedException ie) {
            Thread.currentThread().interrupt();
        } finally {
            executor.shutdownNow(); // ensure thread is reclaimed
        }
    }
}
```

Reasoning to interviewer:

- get(timeout) gives me a clear boundary; on timeout I cancel with mayInterruptIfRunning=true.
- Cooperative interruption requires tasks to check isInterrupted() or handle InterruptedException—I structure the loop to be interruption-friendly.
- Unwrapping ExecutionException surfaces real failure causes for logging/metrics.

Future scope:

- Use CompletableFuture.orTimeout(...) / completeOnTimeout(...) on newer Java for more fluent timeouts.
- Apply exponential backoff/retry on transient failures; skip retries on permanent errors.
- Consider ExecutorService lifecycle hooks or structured concurrency to ensure no task outlives its scope.

Scenario 13: Chain dependent async steps without blocking

In a checkout flow, Step A authorizes payment. Only after it succeeds should Step B create a shipment. Both steps should run asynchronously; no thread should block waiting.

Answer:

I'll use CompletableFuture.thenCompose to model "B after A with A's result." thenCompose flattens the nested future so the pipeline remains async. If either step fails, I attach exceptionally or handle to centralize recovery.

Code:

```
import java.util.concurrent.*;
import java.util.function.Function;

public class CompletableFutureChainDemo {
  private static final ExecutorService pool = Executors.newFixedThreadPool(2);

  static CompletableFuture<String> authorizePayment(String orderId) {
    return CompletableFuture.supplyAsync(() -> {
      sleep(600);
      return "AUTH_OK:" + orderId;
    }, pool);
  }

  static CompletableFuture<String> createShipment(String authToken) {
    return CompletableFuture.supplyAsync(() -> {
      sleep(700);
      return "SHIP_CREATED for " + authToken;
    }, pool);
  }

  public static void main(String[] args) {
    String orderId = "ORD-101";

    CompletableFuture<String> pipeline =
      authorizePayment(orderId)
        .thenCompose((Function<String, CompletableFuture<String>>)
CompletableFutureChainDemo::createShipment)
        .thenApply(shipId -> "Success -> " + shipId)
        .exceptionally(ex -> "Failed -> " + ex.getMessage());

    System.out.println(pipeline.join()); // non-blocking in callers until here
    pool.shutdown();
  }
```

```
   static void sleep(long ms) { try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } }
}
```

Reasoning to interviewer:

- thenCompose is the right choice for dependent stages—B starts only after A completes and its output is available.
- If the tasks were independent (e.g., fetch inventory and pricing), I'd use thenCombine.
- I keep the pipeline non-blocking and only call join() at the edge (e.g., controller boundary) if I must produce a synchronous result.

Future scope:

- Add handle to map failures into domain-specific responses (e.g., reserve stock but mark shipment pending manual review).
- For longer chains, centralize an Executor tuned for I/O vs CPU work to avoid pool starvation.
- Consider virtual threads for simpler blocking code while keeping the same composition semantics.

Scenario 14: Limit concurrent access with permits

Imagine a car parking lot with only 2 parking slots. At any time, only 2 cars should be inside. If a 3rd car arrives, it must wait until one slot is free.

Answer:

This maps perfectly to a Semaphore. A Semaphore starts with a fixed number of permits (slots). Each thread must acquire() before entering, and release() when done. If no permits remain, threads block until one is available.

Code:

```
import java.util.concurrent.Semaphore;

class Car implements Runnable {
   private final Semaphore parking;

   Car(Semaphore parking) {
      this.parking = parking;
   }

   @Override
   public void run() {
      try {
         parking.acquire(); // try to park
         System.out.println(Thread.currentThread().getName() + " parked");
         Thread.sleep(1000); // simulate parking time
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } finally {
         System.out.println(Thread.currentThread().getName() + " leaving");
         parking.release(); // free slot
      }
   }
}

public class ParkingLotDemo {
   public static void main(String[] args) {
      Semaphore parking = new Semaphore(2); // only 2 slots
      for (int i = 1; i <= 5; i++) {
         new Thread(new Car(parking), "Car-" + i).start();
      }
   }
}
```

Reasoning to interviewer:

- Semaphore is a counting lock: it lets me define “up to N threads at once.”
- acquire() blocks when permits run out, ensuring no more than N concurrent users.
- release() must always be in finally to avoid “permits leak.”

Future scope:

- Use tryAcquire(timeout) for time-bound waiting (e.g., car leaves if no slot in 5 sec).
- Fair semaphores (new Semaphore(permits, true)) grant permits FIFO style.
- Semaphores can also be used for resource pools (DB connections, worker slots).

Scenario 15: Heavy computations in parallel with limited threads

Suppose you need to process 10 large image files simultaneously, but if you start 10 threads, the system may thrash. You want to run them in parallel but cap concurrency to 4 at a time.

Answer:

I'll use an ExecutorService with a fixed-size thread pool. Tasks are submitted, and the pool ensures only 4 run at once. Futures let me collect results once done.

Code:

```
import java.util.*;
import java.util.concurrent.*;

class ImageProcessor implements Callable<String> {
   private final int id;

   ImageProcessor(int id) {
      this.id = id;
   }

   @Override
   public String call() throws Exception {
      Thread.sleep(2000); // simulate heavy computation
      return "Image " + id + " processed";
   }
}

public class ImageProcessingDemo {
   public static void main(String[] args) throws Exception {
      ExecutorService pool = Executors.newFixedThreadPool(4); // cap at 4

      List<Future<String>> futures = new ArrayList<>();
      for (int i = 1; i <= 10; i++) {
         futures.add(pool.submit(new ImageProcessor(i)));
      }

      for (Future<String> f : futures) {
         System.out.println(f.get());
      }

      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- Fixed thread pools prevent resource exhaustion — I don't spin up 10 OS threads at once.
- Futures allow me to block for results when I need them, while tasks still run in parallel.
- By tuning pool size to available cores, I balance throughput vs system load.

Future scope:

- Use CompletableFuture for async pipelines (non-blocking result handling).
- Scale with ForkJoinPool or parallel streams if tasks can recursively split.
- Switch to virtual threads in newer Java to simplify concurrency without heavy pooling.

Scenario 16: Many readers, few writers

Think of a stock-price dashboard. Dozens of readers query the latest price constantly, but occasionally a writer updates the price. Readers should run in parallel, but when a writer comes, it must block readers and write safely.

Answer:

This is where ReadWriteLock shines. Multiple readers can hold the readLock concurrently, but only one writer can hold the writeLock, and it excludes readers.

Code:

```
import java.util.concurrent.locks.*;

class StockData {
   private final ReadWriteLock rwLock = new ReentrantReadWriteLock();
   private double price = 100.0;

   public void updatePrice(double newPrice) {
      rwLock.writeLock().lock();
      try {
         price = newPrice;
         System.out.println("Price updated to " + price);
      } finally {
         rwLock.writeLock().unlock();
      }
   }

   public double getPrice() {
      rwLock.readLock().lock();
      try {
         return price;
      } finally {
         rwLock.readLock().unlock();
      }
   }
}

public class StockDashboardDemo {
   public static void main(String[] args) {
      StockData stock = new StockData();

      Thread writer = new Thread(() -> stock.updatePrice(120.5));
      Thread reader1 = new Thread(() -> System.out.println("Reader1 saw: " +
stock.getPrice()));
```

```
        Thread reader2 = new Thread(() -> System.out.println("Reader2 saw: " +
stock.getPrice()));

        writer.start();
        reader1.start();
        reader2.start();
    }
}
```

Reasoning to interviewer:

- readLock allows concurrent reads — perfect for dashboards or caches where reads dominate.
- writeLock blocks everyone else, ensuring no stale/partial writes.
- Always release locks in finally to avoid deadlocks.

Future scope:

- Use StampedLock for optimistic reads — better performance when writes are rare.
- Monitor read/write ratios to tune lock strategy (e.g., snapshot copies if reads are extreme).
- Consider immutable data with atomic swaps to avoid locks when latency is critical.

Scenario 17: Graceful cancellation of long-running tasks

You're running multiple data exports in parallel. If the user clicks "Cancel" or a watchdog hits a timeout, you should stop only the relevant tasks—and do it cleanly so no half-written files or leaked connections remain.

Answer:

I cancel by calling Future.cancel(true), which requests interruption. For this to work, tasks must be written to cooperate: periodically check Thread.currentThread().isInterrupted() and/or handle InterruptedException from blocking calls (sleep, I/O with timeouts). On cancellation, the caller should handle CancellationException from Future.get().

Code:

```
import java.util.concurrent.*;

class InterruptibleExport implements Callable<String> {
  @Override
  public String call() throws Exception {
    try {
      for (int step = 1; step <= 10; step++) {
        // Cooperate with cancellation
        if (Thread.currentThread().isInterrupted()) {
          cleanup();
          return "Cancelled at step " + step;
        }
        // Simulate chunked work that's interruption-friendly
        doChunk(step);      // CPU work
        Thread.sleep(250);   // blocking; throws InterruptedException when cancelled
      }
      finalizeExport();
      return "Completed";
    } catch (InterruptedException ie) {
      // Preserve interrupt status and clean up
      Thread.currentThread().interrupt();
      cleanup();
      return "Cancelled (InterruptedException)";
    }
  }

  private void doChunk(int step) { /* compute/write a chunk */ }
  private void finalizeExport()  { /* close streams, flush, commit */ }
  private void cleanup()       { /* rollback temp files, close resources */ }
}
```

```
public class TaskCancellationDemo {
  public static void main(String[] args) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(2);
    Future<String> f1 = pool.submit(new InterruptibleExport());
    Future<String> f2 = pool.submit(new InterruptibleExport());

    // Let them run a bit, then cancel one (e.g., user cancelled that export)
    Thread.sleep(900);
    f1.cancel(true); // request interruption

    try {
      System.out.println("f1 -> " + f1.get()); // will throw CancellationException
    } catch (CancellationException ce) {
      System.out.println("f1 -> cancelled");
    }
    System.out.println("f2 -> " + f2.get());

    pool.shutdownNow();
  }
}
```

Reasoning to interviewer:

- cancel(true) doesn't "kill" a thread; it sets interrupt status and interrupts blocking calls. The task must check/propagate interruption and clean up.
- I structure work in short chunks and use blocking calls that react to interrupts, so cancellation latency is low.
- I always restore interrupt status when catching InterruptedException, so higher layers can still observe it.

Future scope:

- Centralize timeouts with CompletableFuture.orTimeout / completeOnTimeout.
- Use ScheduledExecutorService for watchdogs that cancel tardy tasks.
- Add idempotent cleanup/rollback so partial work is safe to discard.

Scenario 18: Orchestrating interdependent tasks (DAG style)

You're processing an order: first fetch the customer profile (A). After A finishes, start two tasks in parallel—fetch discounts (B) and fetch inventory (C). When both B and C finish, compute the final quote (D).

Answer:

I model the graph with CompletableFuture: thenCompose for dependent sequencing (B/C after A) and thenCombine/allOf to merge independent results. This keeps everything asynchronous and readable.

Code:

```
import java.util.concurrent.*;
import java.util.function.Function;

public class DependencyManagementDemo {
  private static final ExecutorService io = Executors.newFixedThreadPool(4);

  static CompletableFuture<String> fetchProfile(String orderId) {
    return CompletableFuture.supplyAsync(() -> {
      sleep(300); return "PROFILE(" + orderId + ")";
    }, io);
  }

  static CompletableFuture<String> fetchDiscounts(String profile) {
    return CompletableFuture.supplyAsync(() -> {
      sleep(400); return "DISCOUNT(10%) for " + profile;
    }, io);
  }

  static CompletableFuture<String> fetchInventory(String profile) {
    return CompletableFuture.supplyAsync(() -> {
      sleep(250); return "INVENTORY(OK) for " + profile;
    }, io);
  }

  static String computeQuote(String discounts, String inventory) {
    return "QUOTE{" + discounts + " & " + inventory + "}";
  }

  public static void main(String[] args) {
    String orderId = "ORD-77";

    CompletableFuture<String> pipeline =
      fetchProfile(orderId)
```

```
            .thenCompose((Function<String, CompletableFuture<String>>) profile ->
              // Run B and C in parallel after A
              fetchDiscounts(profile).thenCombine(fetchInventory(profile),
DependencyManagementDemo::computeQuote)
            )
            .exceptionally(ex -> "FAILED -> " + ex.getMessage());

      System.out.println(pipeline.join());
      io.shutdown();
    }

    static void sleep(long ms){ try{ Thread.sleep(ms); } catch(InterruptedException e){
Thread.currentThread().interrupt(); } }
}
```

Reasoning to interviewer:

- thenCompose sequences dependent steps (B/C start only after A).
- thenCombine (or allOf) merges independent branches without blocking threads with get().
- Failures propagate down the chain; I attach one recovery handler for the whole pipeline.

Future scope:

- Add timeouts per stage and retries with backoff for flaky calls.
- Use handle to degrade gracefully (e.g., proceed without discounts).
- Adopt virtual threads to simplify blocking I/O while keeping the same composition.

Scenario 19: Strict task order (A → B → C) without thread juggling

You need heavy jobs to run one after another—image decode, then transform, then upload. They must not overlap, but you still want clean, asynchronous composition.

Answer:

Two clean options:

1. Chain with CompletableFuture (thenRun/thenCompose) for explicit sequencing.
2. Use a single-threaded ExecutorService to serialize tasks automatically (any submitted task runs after the previous one).

Code (CompletableFuture chaining):

```
import java.util.concurrent.*;

public class SequentialExecutionDemo {
  private static final ExecutorService cpu = Executors.newFixedThreadPool(2);

  static CompletableFuture<Void> decode() {
    return CompletableFuture.runAsync(() -> work("decode", 400), cpu);
  }
  static CompletableFuture<Void> transform() {
    return CompletableFuture.runAsync(() -> work("transform", 500), cpu);
  }
  static CompletableFuture<Void> upload() {
    return CompletableFuture.runAsync(() -> work("upload", 300), cpu);
  }

  public static void main(String[] args) {
    CompletableFuture<Void> chain =
      decode()
        .thenCompose(v -> transform())
        .thenCompose(v -> upload())
        .exceptionally(ex -> { System.out.println("Failed: " + ex); return null; });

    chain.join();
    cpu.shutdown();
  }
  static void work(String name, long ms) {
    System.out.println("Start " + name);
    try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
    System.out.println("End " + name);
  }
}
```

Code (Single-threaded executor to serialize automatically):

```
import java.util.concurrent.*;

public class SerializedExecutorDemo {
   public static void main(String[] args) throws Exception {
      ExecutorService single = Executors.newSingleThreadExecutor(); // guarantees
order
      Future<?> f1 = single.submit(() -> step("A"));
      Future<?> f2 = single.submit(() -> step("B")); // runs after A
      Future<?> f3 = single.submit(() -> step("C")); // runs after B
      f1.get(); f2.get(); f3.get();
      single.shutdown();
   }
   static void step(String name){ System.out.println("Step " + name); }
}
```

Reasoning to interviewer:

- thenCompose gives explicit, readable sequencing while staying async.
- A single-threaded executor is the simplest enforcement of strict order—no additional latches or semaphores required.
- I only block at the very edge (join()/get()) if a synchronous boundary needs the final result.

Future scope:

- Wrap the chain in a retry/circuit-breaker layer.
- Use structured concurrency (when available) to scope task lifetimes and cancellation.
- If steps become parallelizable (e.g., multiple transforms), fan-out with thenCombine/allOf between sequential stages.

Scenario 20: Keep the pipeline running when some tasks fail

You're aggregating pricing from multiple providers (A, B, C). Any one provider can flake due to network blips or bad payloads, but the page should still load using whatever data is available (plus sensible fallbacks).

Answer:

I'll launch each provider call as a CompletableFuture and attach per-task error handling using exceptionally, handle, or whenComplete. The key is: failures are contained and converted into defaults, so the overall aggregation can continue and finish normally.

Code:

```
import java.util.concurrent.*;
import java.util.*;

public class ErrorHandlingDemo {
  private static final ExecutorService io = Executors.newFixedThreadPool(4);

  static CompletableFuture<Double> fetchFromA(String sku) {
    return CompletableFuture.supplyAsync(() -> unstable("A"), io)
        .exceptionally(ex -> {
          log("A failed: " + ex.getMessage());
          return 0.0; // fallback
        });
  }

  static CompletableFuture<Double> fetchFromB(String sku) {
    return CompletableFuture.supplyAsync(() -> unstable("B"), io)
        .handle((val, ex) -> ex == null ? val : 0.0); // normalize error -> default
  }

  static CompletableFuture<Double> fetchFromC(String sku) {
    return CompletableFuture.supplyAsync(() -> unstable("C"), io)
        .whenComplete((val, ex) -> { if (ex != null) log("C error: " + ex); });
  }

  static double unstable(String name) {
    if (Math.random() < 0.5) throw new RuntimeException(name + " transient failure");
    return Math.round(500 + Math.random() * 200);
  }

  static void log(String s) { System.out.println(s); }

  public static void main(String[] args) {
    String sku = "SKU-42";
```

```
    CompletableFuture<Double> a = fetchFromA(sku);
    CompletableFuture<Double> b = fetchFromB(sku);
    CompletableFuture<Double> c = fetchFromC(sku).exceptionally(ex -> 0.0);

    // Wait for all, then aggregate safely
    Double total = CompletableFuture.allOf(a, b, c)
      .thenApply(v -> a.join() + b.join() + c.join())
      .join();

    System.out.println("Total price (with fallbacks): " + total);
    io.shutdown();
  }
}
```

Reasoning to interviewer:

- I isolate failures at the leaf futures, converting exceptions to domain defaults (e.g., 0 price or cached value).
- exceptionally maps failure → fallback; handle lets me inspect both success and failure; whenComplete is for side-effects (logging/metrics).
- I still use allOf so the aggregator completes only after every branch settles, but no single failure tears down the pipeline.

Future scope:

- Add per-provider timeouts and limited retries with backoff; skip providers tripping a circuit breaker.
- Record which provider failed to drive alerting and adaptive routing.
- Prefer cached/stale data for better UX when multiple providers fail.

Scenario 21: Fire many independent tasks and resume only when all are done

You render a dashboard with multiple widgets (orders, revenue, traffic). Each widget loads from a different service and finishes at its own pace, but the page should finalize only after all widgets are ready.

Answer:

I'll kick off each widget load with runAsync/supplyAsync and then use CompletableFuture.allOf(...). allOf returns a future that completes when all the inputs complete, irrespective of order. After that, I can safely read each result.

Code:

```
import java.util.concurrent.*;

public class ParallelTasksDemo {
   private static final ExecutorService io = Executors.newFixedThreadPool(4);

   static CompletableFuture<String> loadOrders() {
      return CompletableFuture.supplyAsync(() -> sleepThen("Orders OK", 800), io);
   }
   static CompletableFuture<String> loadRevenue() {
      return CompletableFuture.supplyAsync(() -> sleepThen("Revenue OK", 500), io);
   }
   static CompletableFuture<String> loadTraffic() {
      return CompletableFuture.supplyAsync(() -> sleepThen("Traffic OK", 700), io);
   }

   static String sleepThen(String s, long ms) {
      try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
      return s;
   }

   public static void main(String[] args) {
      CompletableFuture<String> orders  = loadOrders();
      CompletableFuture<String> revenue = loadRevenue();
      CompletableFuture<String> traffic = loadTraffic();

      CompletableFuture<Void> all = CompletableFuture.allOf(orders, revenue,
traffic);
      all.join(); // wait until all finished

      System.out.println(orders.join());
      System.out.println(revenue.join());
```

```
        System.out.println(traffic.join());

        System.out.println("All widgets ready");
        io.shutdown();
    }
}
```

Reasoning to interviewer:

- allOf is the coordination point; it doesn't give combined values, so I call join() on each input future afterward.
- Using a dedicated executor prevents these I/O tasks from starving CPU-bound work.
- If I want to short-circuit on any failure, I can attach exceptionally at each future or wrap allOf with a handler.

Future scope:

- Replace join() with a safe aggregator that substitutes fallbacks on failure (scenario 20 style).
- Use anyOf for "first result wins" semantics (e.g., race mirrors).
- Add tracing/metrics per widget to spot outliers and slow services.

Scenario 22: Prevent starvation when many threads compete

A shared audit writer accepts log batches from many microservices. Under load, a few hot threads keep grabbing the lock repeatedly, while quieter threads starve and never flush their logs.

Answer:

I'll switch the lock to a fair ReentrantLock(true). In fair mode, threads acquire the lock roughly in arrival order (FIFO), giving every waiter a turn and reducing starvation.

Code:

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class FairLogWriter {
   private final Lock lock = new ReentrantLock(true); // fair mode

   public void writeBatch(String who) {
      lock.lock();
      try {
         System.out.println(who + " writing...");
         Thread.sleep(300);
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } finally {
         lock.unlock();
      }
   }
}

public class FairLockDemo {
   public static void main(String[] args) {
      FairLogWriter writer = new FairLogWriter();

      Runnable r = () -> writer.writeBatch(Thread.currentThread().getName());
      for (int i = 1; i <= 5; i++) new Thread(r, "svc-" + i).start();
   }
}
```

Reasoning to interviewer:

- Fair locks trade throughput for predictability. Under high contention, fairness can reduce lock handoff optimizations but ensures every thread progresses.

• If fairness isn't enough, I also remove unbounded retries, add backoff, and reduce lock hold time by moving non-critical work outside the lock.

• I avoid relying on thread priorities for fairness—Java scheduling isn't guaranteed across OSes.

Future scope:

• Consider queues or mailboxes (e.g., BlockingQueue) so producers don't contend on the same lock.

• Use StampedLock/optimistic reads for read-heavy workloads.

• In distributed cases, shift to fair resource schedulers (rate limiters, token buckets) across nodes.

Scenario 23: Threads keep “dodging” each other and never make progress

Two payment workers try to update the same order. Each “politely” steps aside when it sees the other is active, then immediately retries—and they repeat this dance forever. CPU is busy, but nothing actually completes.

Answer:

That’s livelock: threads are active and responsive, but they make zero forward progress because their reactions keep interfering. To avoid livelock I introduce randomness or backoff (wait a little before retrying, and vary that wait), or change the protocol to ensure one side eventually “wins” (e.g., tryLock with increasing delays or a tie-breaker).

Code:

```
import java.util.Random;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

class LivelockSafeService {
   private final ReentrantLock lock = new ReentrantLock();
   private final Random random = new Random();

   // Tries to do work with backoff to avoid livelock
   public boolean doWorkWithBackoff(String name) throws InterruptedException {
      long backoff = 10; // start small
      for (int attempts = 0; attempts < 20; attempts++) {
         if (lock.tryLock(50, TimeUnit.MILLISECONDS)) {
            try {
               System.out.println(name + " acquired lock, doing work");
               Thread.sleep(80); // simulate work
               return true;
            } finally {
               lock.unlock();
            }
         } else {
            // Back off with jitter
            long wait = backoff + random.nextInt((int) backoff);
            System.out.println(name + " backing off " + wait + "ms");
            Thread.sleep(wait);
            backoff = Math.min(backoff * 2, 200); // exponential, with a cap
         }
      }
      return false; // gave up
   }
}
```

```
public class LivelockDemo {
   public static void main(String[] args) {
      LivelockSafeService svc = new LivelockSafeService();

      Thread t1 = new Thread(() -> {
         try { svc.doWorkWithBackoff("T1"); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
      });
      Thread t2 = new Thread(() -> {
         try { svc.doWorkWithBackoff("T2"); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
      });

      t1.start(); t2.start();
   }
}
```

Reasoning to interviewer:

- In livelock, threads keep yielding/reacting, so there's motion but no progress.
- tryLock with randomized/exponential backoff breaks the "lock-step" reaction pattern. Eventually, one thread acquires the lock and finishes.
- A deterministic tie-breaker (e.g., compare thread IDs or order IDs) also guarantees progress without busy behavior.

Future scope:

- Add a max attempt budget and escalate (log, reroute, or serialize the operation).
- Use queue-based designs (BlockingQueue) to avoid competing critical sections.
- In distributed settings, use leader election or unique tokens to enforce a single winner.

Scenario 24: Enforce a hard deadline for work

A report generator sometimes runs long under heavy load. You want a strict 2-second SLA: if a run exceeds it, abort and move on.

Answer:

Submit the task to an ExecutorService and call Future.get(timeout, unit). On TimeoutException, cancel with future.cancel(true) to interrupt the running thread. The task must be interruption-friendly (periodic checks or blocking calls that throw InterruptedException).

Code:

```
import java.util.concurrent.*;

class LongTask implements Callable<String> {
   @Override
   public String call() throws Exception {
      for (int i = 0; i < 10; i++) {
         // cooperate with cancellation
         if (Thread.currentThread().isInterrupted()) throw new
InterruptedException("Cancelled");
         Thread.sleep(600); // chunked work blocks in small slices
      }
      return "Task finished";
   }
}

public class TimeoutHandlingDemo {
   public static void main(String[] args) {
      ExecutorService executor = Executors.newSingleThreadExecutor();
      Future<String> future = executor.submit(new LongTask());

      try {
         String result = future.get(2, TimeUnit.SECONDS); // hard deadline
         System.out.println(result);
      } catch (TimeoutException e) {
         System.out.println("Exceeded time limit, cancelling...");
         future.cancel(true); // interrupt running task
      } catch (ExecutionException e) {
         System.out.println("Task failed: " + e.getCause());
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } finally {
         executor.shutdownNow(); // reclaim threads
      }
```

```
    }
}
```

Reasoning to interviewer:

- get(timeout) defines the SLA at the call site; a timeout leads to cooperative cancellation via cancel(true).
- I slice work into small chunks so the task checks interrupts and stops quickly.
- After cancellation, I don't rely on task cleanup—my task implementation itself must close files/rollback neatly.

Future scope:

- Use CompletableFuture.orTimeout(...) or completeOnTimeout(...) for fluent timeouts.
- Add retries with exponential backoff when timeouts are transient.
- Emit metrics for timeouts to detect regressions and hotspots.

Scenario 25: Handling InterruptedException the right way

A file migration runs for minutes. If the service is shutting down or a deadline expires, you interrupt the worker. You want it to stop promptly and leave the system consistent.

Answer:

Check interruption periodically (CPU loops) and handle InterruptedException from blocking calls. On catching it, perform cleanup and restore the interrupt status via Thread.currentThread().interrupt() before returning, so upstream logic can see it.

Code:

```
class InterruptibleTask implements Runnable {
  @Override
  public void run() {
    try {
      for (int step = 0; step < 100; step++) {
        // CPU-side check
        if (Thread.currentThread().isInterrupted()) {
          System.out.println("Interrupted flag detected at step " + step);
          cleanup();
          return;
        }
        // Simulate blocking I/O; will throw InterruptedException when interrupted
        doBlockingWork(step);
      }
      finalizeWork();
    } catch (InterruptedException e) {
      System.out.println("Interrupted during blocking call, cleaning up...");
      cleanup();
      // restore interrupt so higher layers can act accordingly
      Thread.currentThread().interrupt();
    }
  }

  private void doBlockingWork(int step) throws InterruptedException {
    // placeholder for I/O with timeouts; here we simulate with sleep
    Thread.sleep(200);
  }

  private void cleanup() { /* close streams, rollback temp files */ }
  private void finalizeWork() { /* commit, mark success */ }
}

public class InterruptedTaskDemo {
  public static void main(String[] args) throws InterruptedException {
```

```
        Thread worker = new Thread(new InterruptibleTask(), "migrator");
        worker.start();

        Thread.sleep(1200);       // let it do a few steps
        worker.interrupt();       // request stop
        worker.join();            // wait for clean exit
        System.out.println("Worker stopped cleanly");
    }
}
```

Reasoning to interviewer:

- Interruption is a cooperative signal, not a forced kill. The task must check flags and handle InterruptedException.
- Restoring the interrupt status after catching is important; otherwise, callers won't know it was interrupted.
- Cleanup must be idempotent so repeated interruptions or retries don't corrupt state.

Future scope:

- Standardize cancellation via a common utility (timeouts, backoff, metrics).
- Prefer I/O with built-in timeouts (channels, HTTP clients) so interruption doesn't leave sockets hanging.
- Combine with Future.cancel(true) or ExecutorService.shutdownNow() for coordinated shutdowns across many tasks.

Scenario 26: Multi-phase work where everyone must rendezvous before the next phase

You're running a data pipeline with three workers. Each worker must finish Phase 1 (extract) before anyone starts Phase 2 (transform), and so on for Phase 3 (load). If any worker lags, the others must wait at the end of the phase.

Answer:

I'll use a CyclicBarrier sized to the number of workers. Each worker loops through all phases, does its phase work, then calls barrier.await(). When the last worker arrives, the optional barrier action runs once, the barrier resets automatically, and all workers proceed to the next phase.

Code:

```
import java.util.concurrent.CyclicBarrier;
import java.util.concurrent.BrokenBarrierException;

class MultiPhaseWorker implements Runnable {
  private final int id;
  private final int totalPhases;
  private final CyclicBarrier barrier;

  MultiPhaseWorker(int id, int totalPhases, CyclicBarrier barrier) {
    this.id = id; this.totalPhases = totalPhases; this.barrier = barrier;
  }

  @Override
  public void run() {
    try {
      for (int phase = 1; phase <= totalPhases; phase++) {
        doPhaseWork(phase);
        System.out.printf("T%d reached barrier at phase %d%n", id, phase);
        barrier.await(); // wait for peers
      }
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    } catch (BrokenBarrierException e) {
      System.out.printf("T%d: barrier broken, aborting%n", id);
    }
  }

  private void doPhaseWork(int phase) {
    // simulate real work per phase
    try { Thread.sleep(300 + (long)(Math.random()*200)); } catch
(InterruptedException e) { Thread.currentThread().interrupt(); }
```

```
    }
}

public class CyclicBarrierDemo {
    public static void main(String[] args) {
        int workers = 3, phases = 3;
        CyclicBarrier barrier = new CyclicBarrier(workers,
            () -> System.out.println("-- all workers finished this phase --"));

        for (int i = 1; i <= workers; i++) {
            new Thread(new MultiPhaseWorker(i, phases, barrier), "worker-" + i).start();
        }
    }
}
```

Reasoning to interviewer:

- One fixed set of threads loops across phases—this is the intended CyclicBarrier pattern.
- Barrier action is ideal for “phase-close” logic (e.g., roll up metrics, rotate temp files).
- If one worker fails, the barrier breaks, and others get BrokenBarrierException instead of waiting forever.

Future scope:

- If worker count changes across phases, switch to Phaser (dynamic registration).
- Add timeouts to await() and fail fast if a phase stalls.
- For one-shot rendezvous, CountDownLatch is lighter.

Scenario 27: Many CPU-heavy tasks without melting the machine

You need to run hundreds of expensive computations. Spawning one thread per task thrashes the CPU and GC. You want bounded parallelism and predictable throughput.

Answer:

I'll use a bounded ExecutorService (fixed thread pool sized roughly to CPU cores) with a bounded work queue. This caps concurrency, reuses threads, and avoids resource exhaustion. I'll also set a rejection policy so the system degrades predictably under overload.

Code:

```
import java.util.*;
import java.util.concurrent.*;

class HeavyComputation implements Callable<String> {
   private final int id;
   HeavyComputation(int id) { this.id = id; }
   @Override
   public String call() {
      // CPU-bound work
      double acc = 0;
      for (int i = 0; i < 5_000_000; i++) acc += Math.sin(i);
      return "Task " + id + " done (" + acc + ")";
   }
}

public class BoundedPoolDemo {
   public static void main(String[] args) throws Exception {
      int cores = Runtime.getRuntime().availableProcessors();
      ThreadPoolExecutor pool = new ThreadPoolExecutor(
         cores,              // corePoolSize
         cores,              // maximumPoolSize (fixed)
         0L, TimeUnit.MILLISECONDS,
         new ArrayBlockingQueue<>(50),// bounded queue
         new ThreadPoolExecutor.CallerRunsPolicy() // backpressure on submitter
      );

      List<Future<String>> futures = new ArrayList<>();
      for (int i = 1; i <= 100; i++) futures.add(pool.submit(new HeavyComputation(i)));

      for (Future<String> f : futures) System.out.println(f.get());
      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- Fixed pool ≈ number of cores is a good baseline for CPU-bound tasks.
- Bounded queue + CallerRunsPolicy turns overload into natural backpressure rather than exploding threads.
- Reusing threads cuts creation/destruction overhead and improves cache locality.

Future scope:

- For divide-and-conquer algorithms, use ForkJoinPool or parallel streams.
- Add metrics (queue size, task latency) to tune pool size.
- Consider virtual threads for I/O-bound work; stick to bounded pools for hot CPU loops.

Scenario 28: Server handling lots of requests without one-thread-per-request

Your HTTP server gets bursts of requests. Creating a brand-new thread per request wastes memory and context-switch time. You want to reuse a small, efficient pool.

Answer:

I'll process requests on a thread pool (ExecutorService). Requests are submitted as Runnables/Callables, queued, and executed by pooled threads. This evens out bursts, caps concurrency, and avoids per-request thread creation.

Code:

```
import java.util.concurrent.*;

class RequestHandler implements Runnable {
   private final int id;
   RequestHandler(int id) { this.id = id; }

   @Override
   public void run() {
      // Simulate request work
      try {
         System.out.println(Thread.currentThread().getName() + " handling req " + id);
         Thread.sleep(300);
      } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
   }
}

public class ServerThreadPool {
   public static void main(String[] args) {
      ThreadPoolExecutor pool = new ThreadPoolExecutor(
         3, 3, 0L, TimeUnit.MILLISECONDS,
         new LinkedBlockingQueue<>(100),            // buffer bursts
         r -> { Thread t = new Thread(r, "worker-" + System.nanoTime());
t.setDaemon(true); return t; },
         new ThreadPoolExecutor.AbortPolicy()       // or CallerRunsPolicy for
backpressure
      );

      for (int i = 1; i <= 20; i++) pool.submit(new RequestHandler(i));
      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- Pool threads are reused across many requests; the queue smooths spikes.
- Rejection policy is explicit: abort (fail fast) or caller-runs (throttle upstream).
- Naming threads helps tracing; keeping work small and non-blocking improves throughput.

Future scope:

- Use a ScheduledThreadPoolExecutor for periodic jobs (cleanup, heartbeats).
- For I/O-heavy servers, consider virtual threads (project Loom) to simplify blocking logic with massive concurrency.
- Add graceful shutdown: shutdown() → awaitTermination() → shutdownNow() for clean exits.

Scenario 29: Separate pipelines for high/low priority, served in order

You're building an API gateway that accepts "high-priority" (alerts) and "low-priority" (analytics) requests. Both types should be processed in FIFO order for their own queue, but you want strict control over who runs when—e.g., serve high first while still letting low progress occasionally.

Answer:

I'd use one ReentrantLock with two Conditions—one per priority. Each request enqueues itself and waits on its own condition until a dispatcher (or the producer side) signals it's that priority's turn. Always use await in a loop to re-check the condition (spurious wakeups, lost signals). This gives fine-grained flow control across multiple request classes without juggling multiple locks.

Code:

```
import java.util.ArrayDeque;
import java.util.Queue;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

class PriorityScheduler {
  private final ReentrantLock lock = new ReentrantLock(true); // fair scheduling
  private final Condition highCanRun = lock.newCondition();
  private final Condition lowCanRun  = lock.newCondition();

  private final Queue<Runnable> highQ = new ArrayDeque<>();
  private final Queue<Runnable> lowQ  = new ArrayDeque<>();

  // Policy: serve up to N high items, then 1 low to avoid starvation
  private final int highBurst = 3;
  private int highServedInBurst = 0;

  public void submitHigh(Runnable r) {
    lock.lock();
    try {
      highQ.add(r);
      highCanRun.signal(); // wake a waiting high worker
    } finally {
      lock.unlock();
    }
  }
  public void submitLow(Runnable r) {
    lock.lock();
    try {
      lowQ.add(r);
```

```
        lowCanRun.signal(); // wake a waiting low worker
      } finally {
        lock.unlock();
      }
    }

    // Worker API: blocks until a task of its priority is allowed to run
    public Runnable takeHigh() throws InterruptedException {
      lock.lock();
      try {
        while (!canRunHigh()) highCanRun.await();
        highServedInBurst++;
        return highQ.poll();
      } finally {
        lock.unlock();
      }
    }
    public Runnable takeLow() throws InterruptedException {
      lock.lock();
      try {
        while (!canRunLow()) lowCanRun.await();
        highServedInBurst = 0; // reset after letting low run
        return lowQ.poll();
      } finally {
        lock.unlock();
      }
    }

    private boolean canRunHigh() {
      // high can run if queue non-empty AND either low queue empty or burst not
exhausted
      return !highQ.isEmpty() && (lowQ.isEmpty() || highServedInBurst < highBurst);
    }
    private boolean canRunLow() {
      // low can run if queue non-empty AND (no high items) OR (high burst completed)
      return !lowQ.isEmpty() && (highQ.isEmpty() || highServedInBurst >= highBurst);
    }
}

public class MultipleConditionsDemo {
    public static void main(String[] args) {
      PriorityScheduler sch = new PriorityScheduler();

      // High workers
      for (int i = 0; i < 2; i++) {
        new Thread(() -> {
          while (true) {
```

```
            try {
              Runnable job = sch.takeHigh();
              job.run();
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); break;
}
          }
        }, "high-worker-" + i).start();
      }

      // Low workers
      new Thread(() -> {
        while (true) {
          try {
            Runnable job = sch.takeLow();
            job.run();
          } catch (InterruptedException e) { Thread.currentThread().interrupt(); break; }
        }
      }, "low-worker").start();

      // Submit jobs
      for (int i = 1; i <= 5; i++) sch.submitHigh(() ->
System.out.println(Thread.currentThread().getName() + " handled HIGH"));
      for (int i = 1; i <= 5; i++) sch.submitLow(() ->
System.out.println(Thread.currentThread().getName() + " handled low"));
    }
}
```

Reasoning to interviewer:

- One ReentrantLock + multiple Conditions splits wait-sets by concern (high vs low). That's cleaner than one condition with if/else spaghetti.
- Always await inside a loop so threads re-check the policy after being signaled.
- The burst logic shows how to prevent starvation without abandoning priority.

Future scope:

- Replace burst counters with time-based fairness (serve low if it has waited > X ms).
- Use signalAll() when multiple workers per priority and simple policy; otherwise target with signal() to reduce thundering herds.
- Metrics: average wait time per priority to tune the policy.

Scenario 30: Two-way money transfers sometimes freeze the system (deadlock risk)

Two transfer threads run: T1 moves money A→B; T2 moves B→A. T1 locks A then waits for B; T2 locks B then waits for A. Both wait forever—your system appears hung.

Answer:

Classic deadlock via circular wait. I prevent it by enforcing a strict global lock order: always lock the account with the smaller id first, then the larger. Alternatively, use tryLock with timeout + backoff to detect and avoid deadlock, but deterministic ordering is simplest and fastest.

Code (fixed lock order with ReentrantLock):

```
import java.util.concurrent.locks.ReentrantLock;

class BankAccount {
   final long id;
   private int balance;
   private final ReentrantLock lock = new ReentrantLock();

   BankAccount(long id, int balance) { this.id = id; this.balance = balance; }
   void lock()   { lock.lock(); }
   void unlock() { lock.unlock(); }

   void add(int amt) { balance += amt; }
   boolean has(int amt) { return balance >= amt; }
   int get() { return balance; }
}

class Transfers {
   public static void transfer(BankAccount from, BankAccount to, int amount) {
      BankAccount first  = from.id < to.id ? from : to;
      BankAccount second = from.id < to.id ? to   : from;

      first.lock();
      try {
         second.lock();
         try {
            if (!from.has(amount)) throw new IllegalStateException("Insufficient funds");
            from.add(-amount);
            to.add(amount);
            System.out.printf("Moved %d from %d to %d (balances: %d, %d)%n",
                  amount, from.id, to.id, from.get(), to.get());
         } finally {
            second.unlock();
```

```
        }
      } finally {
        first.unlock();
      }
    }
  }

public class BankDeadlockSafeDemo {
  public static void main(String[] args) {
    BankAccount a = new BankAccount(1, 1000);
    BankAccount b = new BankAccount(2, 1000);

    Thread t1 = new Thread(() -> Transfers.transfer(a, b, 200));
    Thread t2 = new Thread(() -> Transfers.transfer(b, a, 150));
    t1.start(); t2.start();
  }
}
```

Reasoning to interviewer:

- Deadlock requires four conditions (mutual exclusion, hold-and-wait, no preemption, circular wait). Breaking circular wait via lock ordering is the cheapest fix.
- The moment every transfer takes locks in the same global order, you eliminate the cycle.
- Avoid calling synchronized methods on other account objects from inside a synchronized block (nested monitor locks with no order = deadlock bait).

Future scope:

- If accounts can be the same object, short-circuit (no locking twice).
- For multi-account transfers (A→B→C), sort all accounts and lock in ascending order.
- As a fallback under heterogeneous code paths, use tryLock + randomized backoff to detect and retry instead of deadlocking.

Scenario 31: Mixed-duration jobs with limited workers

A video platform needs to generate thumbnails, transcode clips, and extract subtitles. Each job takes a different amount of time. You've got only a few worker threads, and you want those threads to stay busy—no one should sit idle just because a long job is blocking a result.

Answer:

I'll run everything on a bounded ExecutorService so we cap concurrency and reuse threads. To avoid "head-of-line blocking" when collecting results, I'll consume completions as they arrive (via ExecutorCompletionService) instead of calling get() in submission order. This keeps workers busy and makes the system responsive to short jobs finishing early.

Code:

```
import java.util.*;
import java.util.concurrent.*;

class VariableDurationTask implements Callable<String> {
   private final String name;
   private final long ms; // variable duration

   VariableDurationTask(String name, long ms) { this.name = name; this.ms = ms; }

   @Override
   public String call() throws Exception {
      Thread.sleep(ms); // simulate work
      return name + " completed in " + ms + "ms";
   }
}

public class VariableTaskThreadPool {
   public static void main(String[] args) throws Exception {
      ExecutorService pool = Executors.newFixedThreadPool(3); // limited workers
      ExecutorCompletionService<String> ecs = new
ExecutorCompletionService<>(pool);

      List<Callable<String>> jobs = List.of(
         new VariableDurationTask("thumb-A", 300),
         new VariableDurationTask("transcode-X", 1500),
         new VariableDurationTask("subs-M", 700),
         new VariableDurationTask("thumb-B", 200),
         new VariableDurationTask("transcode-Y", 1200)
      );
```

```
        jobs.forEach(ecs::submit);

        for (int i = 0; i < jobs.size(); i++) {
            System.out.println(ecs.take().get()); // process as each finishes
        }

        pool.shutdown();
    }
}
```

Reasoning to interviewer:

- Fixed pool avoids spawning one thread per task and keeps CPU/memory predictable.
- Completion-first collection prevents a slow task from delaying all subsequent results.
- I keep tasks independent and short enough that thread reuse is effective; for CPU-bound work, size the pool ≈ number of cores.

Future scope:

- Track queue length and latency; apply backpressure (bounded queue + CallerRunsPolicy).
- For CPU-heavy divide-and-conquer, consider ForkJoinPool/parallel streams.
- For I/O-heavy work, consider virtual threads to simplify blocking without huge pools.

Scenario 32: Cap concurrent client connections

Your TCP server can handle only 3 simultaneous connections to an expensive backend. Any extra clients should wait until a slot frees up.

Answer:

Use a Semaphore with 3 permits. Each handler acquires a permit before beginning and releases it on completion. Threads block (or time out) when no permits are available, which naturally limits concurrency.

Code:

```
import java.util.concurrent.Semaphore;

class Server {
   private final Semaphore permits = new Semaphore(3, true); // fair to reduce
starvation

   public void handleClient(String clientName) {
      boolean acquired = false;
      try {
         acquired = permits.tryAcquire(); // or tryAcquire(timeout, unit)
         if (!acquired) {
            System.out.println(clientName + " waiting for a free slot");
            permits.acquire(); // block until available
         }
         System.out.println(clientName + " connected");
         Thread.sleep(1000); // simulate work
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } finally {
         if (acquired || permits.availablePermits() < 3) {
            System.out.println(clientName + " disconnected");
            permits.release();
         }
      }
   }
}

public class SemaphoreServerDemo {
   public static void main(String[] args) {
      Server server = new Server();
      for (int i = 1; i <= 6; i++) {
         final String name = "Client-" + i;
         new Thread(() -> server.handleClient(name)).start();
      }
```

```
    }
}
```

Reasoning to interviewer:

- A counting semaphore models a pool of scarce resources (connections).
- Fair mode approximates FIFO admission to avoid starvation under load.
- tryAcquire(timeout) gives clients a deadline, useful for fail-fast behavior.

Future scope:

- Track active sessions; refuse low-priority clients when load is high.
- Replace sleep with real I/O that supports timeouts/cancellation.
- Combine with a connection pool and per-tenant rate limits.

Scenario 33: Start many tasks, handle results the moment they're ready

You launch multiple enrichment calls for a search results page. Some finish fast, others slow. You want to render partial sections as soon as data arrives.

Answer:

Pair your executor with an ExecutorCompletionService. Submit all tasks, then pull completed Futures using take()/poll(). This yields results in completion order, not submission order.

Code:

```
import java.util.concurrent.*;

class SimpleTask implements Callable<String> {
   private final String name; private final long ms;
   SimpleTask(String name, long ms) { this.name = name; this.ms = ms; }
   @Override public String call() throws Exception { Thread.sleep(ms); return name + "
finished"; }
}

public class CompletionServiceDemo {
   public static void main(String[] args) throws Exception {
      ExecutorService pool = Executors.newFixedThreadPool(3);
      ExecutorCompletionService<String> ecs = new
ExecutorCompletionService<>(pool);

      ecs.submit(new SimpleTask("Task-1", 900));
      ecs.submit(new SimpleTask("Task-2", 300));
      ecs.submit(new SimpleTask("Task-3", 600));

      for (int i = 0; i < 3; i++) {
         Future<String> done = ecs.take();   // next finished
         System.out.println(done.get());     // process immediately
      }
      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- Completion service decouples execution from completion order, improving perceived latency.

- It’s still backed by your chosen pool size, so concurrency stays bounded.
- Add per-task error handling (get with try/catch) to keep the loop robust when a task fails.

Future scope:

- Use poll(timeout) to avoid indefinite blocking and surface slow tasks.
- Mix with CompletableFuture to compose results and add fallbacks.
- Emit metrics (task duration, queue wait) to spot hotspots and tune pool size.

Scenario 34: Producer hands off data; consumer waits until it's ready

Imagine a metrics pipeline: a sensor thread produces readings; a reporter thread publishes them. If no reading exists yet, the reporter must wait; once produced, it should wake up and consume.

Answer:

I'll coordinate the two threads with a shared monitor using wait()/notifyAll(). The producer sets data and signals; the consumer waits in a loop until a flag says data is available. I use a while guard (not if) to handle spurious wakeups and preserve correctness under bursts.

Code:

```
class SharedBuffer {
  private Integer data;
  private boolean available = false;

  public synchronized void produce(int value) throws InterruptedException {
    while (available) {        // buffer full -> wait
      wait();
    }
    data = value;
    available = true;
    System.out.println("Produced: " + data);
    notifyAll();               // wake consumers
  }

  public synchronized int consume() throws InterruptedException {
    while (!available) {       // buffer empty -> wait
      wait();
    }
    int val = data;
    available = false;
    System.out.println("Consumed: " + val);
    notifyAll();               // wake producers
    return val;
  }
}

public class ProducerConsumerDemo {
  public static void main(String[] args) {
    SharedBuffer buf = new SharedBuffer();

    Thread producer = new Thread(() -> {
      for (int i = 1; i <= 5; i++) {
```

```
            try { buf.produce(i); Thread.sleep(150); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
          }
        }, "producer");

        Thread consumer = new Thread(() -> {
          for (int i = 1; i <= 5; i++) {
            try { buf.consume(); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
          }
        }, "consumer");

        producer.start(); consumer.start();
      }
}
```

Reasoning to interviewer:

- wait()/notifyAll() must be called while holding the same monitor (synchronized on the same object).
- The while loop re-checks the predicate after waking—critical for spurious wakeups and avoiding lost-notify bugs.
- I signal with notifyAll() here because multiple waiters or role flips (produce/consume) can otherwise lose a wakeup.

Future scope:

- Replace with BlockingQueue (e.g., ArrayBlockingQueue) to avoid hand-rolled coordination.
- Add multiple producers/consumers and bounded capacity to apply backpressure.
- Add timeouts and metrics to detect stalls.

Scenario 35: Threads grab different locks and end up waiting forever

Two maintenance tasks each need resources A and B. Task1 locks A then tries B; Task2 locks B then tries A. Under load, they can wait indefinitely—nothing progresses.

Answer:

That's deadlock: circular wait between threads. I prevent it by enforcing a global lock order (e.g., always lock the lower-id resource first), or by using tryLock with timeouts and backing off if I can't get both quickly.

Code (fixed global order + tryLock timeout):

```
import java.util.concurrent.TimeUnit;
import java.util.concurrent.locks.ReentrantLock;

class Resource {
   final int id;
   final ReentrantLock lock = new ReentrantLock();
   Resource(int id) { this.id = id; }
}

class ResourceManager {
   public boolean useBoth(Resource a, Resource b) throws InterruptedException {
      Resource first  = a.id < b.id ? a : b;   // global ordering
      Resource second = a.id < b.id ? b : a;

      if (!first.lock.tryLock(200, TimeUnit.MILLISECONDS)) return false;
      try {
         if (!second.lock.tryLock(200, TimeUnit.MILLISECONDS)) return false;
         try {
            System.out.println("Acquired " + first.id + " & " + second.id);
            // do work
            return true;
         } finally {
            second.lock.unlock();
         }
      } finally {
         if (first.lock.isHeldByCurrentThread()) first.lock.unlock();
      }
   }
}

public class DeadlockAvoidDemo {
   public static void main(String[] args) {
      Resource A = new Resource(1), B = new Resource(2);
      ResourceManager rm = new ResourceManager();
```

```
    Runnable r1 = () -> { try { rm.useBoth(A, B); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } };
    Runnable r2 = () -> { try { rm.useBoth(B, A); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } };

    new Thread(r1, "t1").start();
    new Thread(r2, "t2").start();
  }
}
```

Reasoning to interviewer:

- Deadlock needs four conditions (mutual exclusion, hold-and-wait, no preemption, circular wait). Breaking circular wait via ordering is simplest and fastest.
- tryLock with timeout plus retry/backoff gives resilience against unexpected paths violating the order.

Future scope:

- For N resources, sort all targets, then lock in ascending order.
- Include logging/metrics on lock acquisition times to spot contention hot spots.
- Consider lock-free or staged pipelines to minimize overlapping lock scopes.

Scenario 36: Many quick tasks—don't spawn a thread per task

A web crawler schedules thousands of short parsing tasks. Creating/destroying a thread for each is wasteful and trashes caches.

Answer:

Use an ExecutorService (thread pool). Threads are created once and reused for many tasks. A fixed-size pool caps concurrency; a cached pool grows and shrinks for bursty, I/O-heavy workloads.

Code:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolDemo {
   public static void main(String[] args) {
      ExecutorService pool = Executors.newFixedThreadPool(3); // cap concurrency

      for (int i = 0; i < 5; i++) {
         pool.submit(() -> {
            System.out.println("Running on " + Thread.currentThread().getName());
            try { Thread.sleep(200); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
         });
      }

      pool.shutdown(); // graceful stop after tasks finish
   }
}
```

Reasoning to interviewer:

- Pools amortize thread creation cost and smooth bursts via a task queue.
- Fixed vs cached depends on CPU-bound vs I/O-bound: fixed ~ cores for CPU work; cached for I/O with many waits.
- Always shut down pools to avoid thread leaks.

Future scope:

- For backpressure, use a bounded queue and a rejection policy (CallerRunsPolicy).
- Expose pool metrics (active count, queue depth) and tune.

• Consider virtual threads for massive I/O concurrency with simple blocking code.

Scenario 37: Don’t start the app until all subsystems are ready

At startup, you spawn workers to warm caches, load configs, and establish DB connections. The main thread should proceed only when all those steps finish.

Answer:

CountDownLatch fits perfectly. Initialize with the number of init tasks. Each worker countDown()s when done; the main thread calls await() to block until the count reaches zero.

Code:

```
import java.util.concurrent.CountDownLatch;

class InitWorker implements Runnable {
   private final CountDownLatch latch;
   private final String name;
   InitWorker(String name, CountDownLatch latch) { this.name = name; this.latch =
latch; }

   @Override public void run() {
      try {
         System.out.println(name + " initializing...");
         Thread.sleep(300); // simulate work
         System.out.println(name + " ready");
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } finally {
         latch.countDown();
      }
   }
}

public class CountDownLatchDemo {
   public static void main(String[] args) throws InterruptedException {
      CountDownLatch latch = new CountDownLatch(3);

      new Thread(new InitWorker("Cache", latch)).start();
      new Thread(new InitWorker("Config", latch)).start();
      new Thread(new InitWorker("Database", latch)).start();

      latch.await(); // main waits here
      System.out.println("All initialization completed. Starting server...");
   }
}
```

Reasoning to interviewer:

- CountDownLatch is one-shot: once it hits zero, it can't be reused (unlike CyclicBarrier).
- The latch count should reflect required components only—optional ones can signal separately.
- Always countDown() in a finally to avoid hanging the main thread if a worker fails.

Future scope:

- For repeated phase gates, use CyclicBarrier/Phaser.
- Add timeouts to await() and fail fast if a subsystem stalls.
- Feed latch progress into health checks so orchestration can observe readiness.

Scenario 38: Multi-phase workers that must rendezvous at each step

You're coordinating three analytics workers that must all finish Phase 1 before any starts Phase 2, and so on. If one runs late, the others should wait at a "meeting point" and then move together.

Answer:

I'll use a CyclicBarrier with the party count set to the number of workers. Each worker does its phase work, calls barrier.await() to wait for peers, and only then continues. The barrier is reusable, so the same instance gates every phase. A barrier action can run once per phase to perform "phase-close" logic (e.g., flush metrics).

Code:

```
import java.util.concurrent.BrokenBarrierException;
import java.util.concurrent.CyclicBarrier;

class PhaseWorker implements Runnable {
  private final CyclicBarrier barrier;
  private final int totalPhases;
  private final int id;

  PhaseWorker(int id, CyclicBarrier barrier, int totalPhases) {
    this.id = id; this.barrier = barrier; this.totalPhases = totalPhases;
  }

  @Override
  public void run() {
    try {
      for (int phase = 1; phase <= totalPhases; phase++) {
        doPhaseWork(phase);
        System.out.printf("T%d reached barrier for phase %d%n", id, phase);
        barrier.await(); // wait until all arrive
      }
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    } catch (BrokenBarrierException e) {
      System.out.printf("T%d: barrier broken; aborting%n", id);
    }
  }

  private void doPhaseWork(int phase) {
    try { Thread.sleep(200 + (long)(Math.random() * 300)); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
  }
}
```

```
public class CyclicBarrierDemo {
   public static void main(String[] args) {
      int workers = 3, phases = 3;
      CyclicBarrier barrier = new CyclicBarrier(workers,
         () -> System.out.println("-- all threads completed this phase --"));

      for (int i = 1; i <= workers; i++) {
         new Thread(new PhaseWorker(i, barrier, phases), "worker-" + i).start();
      }
   }
}
```

Reasoning to interviewer:

- CyclicBarrier is ideal when a fixed set of threads repeats the same rendezvous across phases.
- await() blocks each thread until the last arrives; then everyone proceeds and the barrier resets automatically.
- If a thread fails/interrupts, the barrier breaks; peers get BrokenBarrierException, so nothing hangs silently.

Future scope:

- If the number of participants can change between phases, switch to Phaser.
- Add a timeout to await() to fail fast on stuck phases.
- Put aggregation or logging in the barrier action to run once per phase.

Scenario 39: Speed up independent element processing over a big list

You have a large list of orders, and each can be scored independently. You want to use all cores without writing thread plumbing.

Answer:

I'll use Java 8 parallel streams. parallelStream() splits work across the common ForkJoinPool, typically leveraging available cores. I'll keep operations stateless and side-effect-free, and use terminal ops/collectors that are safe under parallelism.

Code:

```
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.*;

public class ParallelStreamDemo {
   public static void main(String[] args) {
      List<Integer> orders = IntStream.rangeClosed(1, 10_000).boxed().toList();

      // Example 1: compute sum of squares (numeric reduce is naturally parallel-
friendly)
      long sumOfSquares = orders.parallelStream()
                        .mapToLong(o -> (long)o * o)
                        .sum();
      System.out.println("Sum of squares = " + sumOfSquares);

      // Example 2: produce a result list (note: result order is not guaranteed)
      List<Integer> scores = orders.parallelStream()
                         .map(ParallelStreamDemo::score)
                         .collect(toList());
      System.out.println("Scored " + scores.size() + " orders");

      // If you need encounter order preserved in the terminal stage:
      orders.parallelStream()
          .map(ParallelStreamDemo::score)
          .forEachOrdered(s -> { /* emit in original order */ });
   }

   static int score(int orderId) {
      // pretend CPU-bound scoring
      return (orderId * 31) & 1023;
   }
}
```

Reasoning to interviewer:

- Parallel streams parallelize map/filter/reduce across the common ForkJoinPool with work-stealing—great for CPU-bound, independent operations.
- Avoid shared mutable state; prefer pure functions and built-in collectors.
- forEach on a parallel stream is unordered; use forEachOrdered when you need deterministic output order (with some performance cost).

Future scope:

- For different parallelism than the common pool, submit the stream pipeline to a custom ForkJoinPool (e.g., pool.submit(() -> stream.parallel().collect(...)).join()).
- If tasks are I/O-bound or include blocking calls, consider virtual threads or CompletableFuture to avoid starving the common pool.
- Measure: parallel overhead can outweigh benefits for tiny datasets—use thresholds to switch between sequential and parallel.

Scenario 40: Ensure JVM waits for critical work before exit

Your file-import tool spawns a worker thread to parse a large CSV. Sometimes the JVM exits before parsing finishes because the main thread ends early.

Answer:

I'll start the worker as a non-daemon thread and use join() on it from main (or a coordinator) so the JVM won't exit until the worker completes. If I truly need a background thread, I'll keep it daemon but still coordinate shutdown via ExecutorService and awaitTermination().

Code:

```
class CsvWorker extends Thread {
   public CsvWorker() { setName("csv-worker"); /* non-daemon by default */ }
   @Override public void run() { parseCsv(); }
   private void parseCsv() { /* long-running parse */ }
}

public class JoinDemo {
   public static void main(String[] args) throws InterruptedException {
      Thread worker = new CsvWorker();
      worker.start();
      worker.join(); // ensure completion before JVM exit
      System.out.println("Import finished, exiting cleanly");
   }
}
```

Reasoning to interviewer:

- Daemon threads don't keep the JVM alive; non-daemon + join() does.
- join() provides a simple lifecycle barrier without busy-waiting or sleeps.
- For multiple tasks, ExecutorService.shutdown() + awaitTermination() scales better than manual joins.

Future scope:

- Add timeouts to join() and a fallback cancel if a thread is stuck.
- Use structured shutdown hooks to flush logs and close connections.
- Prefer ExecutorService over raw threads for pooled, observable lifecycles.

Scenario 41: Class already extends a base type but must run concurrently

You have a ReportGenerator that extends BaseService. You also need to execute it on a thread and identify each run in logs.

Answer:

I'll implement Runnable (not extend Thread) so I can still extend BaseService. I'll name threads via a ThreadFactory to get clean, searchable logs.

Code:

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

class ReportGenerator extends BaseService implements Runnable {
   private final String reportId;
   ReportGenerator(String reportId) { this.reportId = reportId; }
   @Override public void run() { generate(reportId); }
   private void generate(String id) { /* heavy compute */ }
}
class BaseService { /* existing base behavior */ }

class NamedFactory implements ThreadFactory {
   private final AtomicInteger seq = new AtomicInteger(1);
   public Thread newThread(Runnable r) {
      Thread t = new Thread(r);
      t.setName("report-worker-" + seq.getAndIncrement());
      t.setUncaughtExceptionHandler((th, ex) ->
         System.err.println("Uncaught in " + th.getName() + ": " + ex));
      return t;
   }
}

public class RunnableWithFactoryDemo {
   public static void main(String[] args) {
      ExecutorService pool = Executors.newFixedThreadPool(2, new NamedFactory());
      pool.submit(new ReportGenerator("R-101"));
      pool.submit(new ReportGenerator("R-202"));
      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- Runnable decouples task from execution and preserves single inheritance.
- ThreadFactory standardizes naming and exception handling.
- Using a pool avoids one-thread-per-task overhead.

Future scope:

- Switch to Callable<T> if I need results or checked exceptions.
- Propagate MDC (logging context) per task for traceability.
- Move to virtual threads when blocking I/O dominates.

Scenario 42: Prevent inconsistent balance with synchronization

Two ATM withdrawals hit the same account concurrently; sometimes the balance briefly goes negative.

Answer:

I'll protect the read–modify–write with a private lock (monitor or ReentrantLock). Keep the critical section minimal: only the balance mutation is locked.

Code:

```
import java.util.concurrent.locks.ReentrantLock;

class Account {
   private final ReentrantLock lock = new ReentrantLock();
   private int balance;

   Account(int initial) { this.balance = initial; }

   public boolean withdraw(int amt) {
      lock.lock();
      try {
         if (amt <= 0) throw new IllegalArgumentException();
         if (balance < amt) return false;
         balance -= amt;
         return true;
      } finally {
         lock.unlock();
      }
   }

   public int snapshot() {
      lock.lock();
      try { return balance; }
      finally { lock.unlock(); }
   }
}
```

Reasoning to interviewer:

- A private lock prevents outside code from interfering (no locking on this).
- Only mutate inside the lock; validate inputs outside to reduce contention.
- ReentrantLock gives me tryLock, fairness, and interruptible acquisition if needed.

Future scope:

- If reads dominate, shift to ReadWriteLock/StampedLock.
- For distributed correctness, pair with DB row locks or optimistic versions.
- Expose metrics for lock wait time to detect contention.

Scenario 43: Consumer sometimes waits forever despite data being produced

You implemented producer/consumer with wait()/notify(), but occasionally the consumer hangs.

Answer:

Common bugs: using if instead of while around wait(), signaling the wrong monitor, or using notify with multiple waiters. I'll guard with while and use notifyAll, or switch to BlockingQueue.

Code:

```
import java.util.LinkedList;
import java.util.Queue;

class SafeBuffer<T> {
   private final Queue<T> q = new LinkedList<>();
   private final int cap = 3;

   public synchronized void put(T item) throws InterruptedException {
      while (q.size() == cap) wait();
      q.add(item);
      notifyAll(); // wake consumers
   }
   public synchronized T take() throws InterruptedException {
      while (q.isEmpty()) wait();
      T v = q.remove();
      notifyAll(); // wake producers
      return v;
   }
}
```

Reasoning to interviewer:

- while re-check protects against spurious wakeups and missed signals.
- notifyAll avoids waking the wrong role when multiple producers/consumers exist.
- A BlockingQueue is the safer high-level primitive.

Future scope:

- Replace with ArrayBlockingQueue/LinkedBlockingQueue.
- Add timeouts for take/put to avoid indefinite stalls.
- Introduce backpressure by bounding capacity and measuring queue depth.

Scenario 44: Avoid deadlock in two-account transfer

Two transfers run simultaneously: A→B and B→A. Threads sometimes freeze.

Answer:

I'll take locks in a strict global order (by account id). That breaks circular wait and removes the deadlock condition.

Code:

```
import java.util.concurrent.locks.ReentrantLock;

class BankAccount {
   final long id; final ReentrantLock lock = new ReentrantLock();
   private int bal;
   BankAccount(long id, int bal) { this.id=id; this.bal=bal; }
   void add(int d){ bal+=d; }
   int get(){ return bal; }
}

class SafeTransfer {
   static void transfer(BankAccount from, BankAccount to, int amt) {
      BankAccount first  = from.id < to.id ? from : to;
      BankAccount second = from.id < to.id ? to   : from;

      first.lock.lock();
      try {
         second.lock.lock();
         try {
            if (from.get() < amt) throw new IllegalStateException("Insufficient");
            from.add(-amt); to.add(amt);
         } finally { second.lock.unlock(); }
      } finally { first.lock.unlock(); }
   }
}
```

Reasoning to interviewer:

- Deadlock needs circular wait; global ordering removes the cycle.
- tryLock + timeout can be added as a defensive fallback.
- Keep lock scope tight; never call out to user code while holding locks.

Future scope:

- For N accounts per transfer, lock them sorted by id.
- Add audit logs/metrics for long lock waits.
- Consider optimistic concurrency with retry for scalability.

Scenario 45: Prevent starvation under heavy contention

A hot spot lock is usually grabbed by a busy thread; quieter threads rarely progress.

Answer:

Switch to a fair ReentrantLock(true) so acquisition is roughly FIFO. I'll also minimize time inside the lock and avoid lock reentrancy where possible.

Code:

```
import java.util.concurrent.locks.ReentrantLock;

class FairHotspot {
   private final ReentrantLock lock = new ReentrantLock(true); // fair
   private int state;

   public void update() {
      lock.lock();
      try {
         state++; // minimal work
      } finally {
         lock.unlock();
      }
   }
}
```

Reasoning to interviewer:

- Fairness trades throughput for predictability; useful when starvation is unacceptable.
- Starvation can also be reduced by backoff (don't spin), and by sharding locks to reduce contention.
- Thread priorities are not portable—don't rely on them.

Future scope:

- Replace shared state with per-thread buffers + periodic merge.
- Use queue-based handoff (e.g., BlockingQueue) instead of a hot lock.
- Instrument lock contention and adapt dynamically (shard factor, fairness).

Scenario 46: Run recurring tasks reliably

You need to refresh a cache every minute and publish a heartbeat every 5 seconds.

Answer:

Use ScheduledThreadPoolExecutor. Choose scheduleAtFixedRate for regular cadence (catching up after delays) or scheduleWithFixedDelay to wait a fixed delay after each run. Handle exceptions so the task doesn't silently die.

Code:

```
import java.util.concurrent.*;

public class SchedulerDemo {
  public static void main(String[] args) {
    ScheduledExecutorService ses = Executors.newScheduledThreadPool(2);

    Runnable cacheRefresh = () -> {
      try { /* refresh cache */ }
      catch (Exception e) { System.err.println("Cache refresh failed: " + e); }
    };
    Runnable heartbeat = () -> System.out.println("beat " +
System.currentTimeMillis());

    ses.scheduleAtFixedRate(cacheRefresh, 0, 1, TimeUnit.MINUTES);
    ses.scheduleWithFixedDelay(heartbeat, 0, 5, TimeUnit.SECONDS);
    // add shutdown hook in real apps
  }
}
```

Reasoning to interviewer:

- Fixed-rate is clock-based; fixed-delay waits after completion.
- Uncaught exceptions cancel scheduled tasks—wrap bodies with try/catch.
- A pool >1 prevents one long task from delaying others.

Future scope:

- Use CompletableFuture inside scheduled tasks to compose async subtasks.
- Apply jitter to avoid thundering herds across instances.
- Add awaitTermination on shutdown for graceful stop.

Scenario 47: Fan-out to multiple services and aggregate safely

A product page needs price, stock, and reviews from different services. Any one may fail, but the page should still render.

Answer:

Kick off independent calls with CompletableFuture.supplyAsync. Use per-branch exceptionally/handle to convert failures to defaults. Wait with allOf, then read results with join().

Code:

```
import java.util.concurrent.*;

public class FanoutAggregateDemo {
  private static final ExecutorService io = Executors.newFixedThreadPool(6);

  static CompletableFuture<Double> price(String sku) {
    return CompletableFuture.supplyAsync(() -> fetchPrice(sku), io)
        .exceptionally(ex -> 0.0);
  }
  static CompletableFuture<Integer> stock(String sku) {
    return CompletableFuture.supplyAsync(() -> fetchStock(sku), io)
        .handle((v, ex) -> ex == null ? v : 0);
  }
  static CompletableFuture<Double> rating(String sku) {
    return CompletableFuture.supplyAsync(() -> fetchRating(sku), io)
        .exceptionally(ex -> 0.0);
  }

  public static void main(String[] args) {
    var p = price("SKU-9");
    var s = stock("SKU-9");
    var r = rating("SKU-9");

    CompletableFuture<Void> all = CompletableFuture.allOf(p, s, r);
    all.join();
    System.out.printf("price=%.2f stock=%d rating=%.1f%n", p.join(), s.join(), r.join());
    io.shutdown();
  }

  // mocks
  static double fetchPrice(String s){ return 999.0; }
  static int fetchStock(String s){ return 12; }
  static double fetchRating(String s){ return 4.6; }
}
```

Reasoning to interviewer:

- Handle errors at the leaf futures so the aggregator doesn't blow up.
- allOf coordinates completion; join() is safe afterward.
- A dedicated I/O pool prevents starving CPU-bound work.

Future scope:

- Add orTimeout per branch; apply retries with backoff for transient failures.
- Use anyOf for "first result wins" (e.g., mirror endpoints).
- Migrate to virtual threads for simpler blocking I/O with huge fan-out.

Scenario 48: Producer–consumer with backpressure and multiple consumers

You're ingesting events from Kafka and pushing them to 3 different indexer threads. If producers are faster than consumers, memory grows and GC thrashes. You want built-in coordination and backpressure without hand-rolling wait/notify.

Answer:

I'll use a bounded BlockingQueue (e.g., ArrayBlockingQueue) as the handoff buffer. Producers put() (blocking when full), consumers take() (blocking when empty). This automatically throttles producers and wakes consumers without manual wait/notify code.

Code:

```
import java.util.concurrent.*;

public class BlockingQueuePipeline {
  public static void main(String[] args) {
    ArrayBlockingQueue<String> queue = new ArrayBlockingQueue<>(100); //
bounded = backpressure
    ExecutorService pool = Executors.newFixedThreadPool(4);

    // Producer
    pool.submit(() -> {
      for (int i = 1; i <= 500; i++) {
        queue.put("event-" + i);          // blocks if full
      }
      queue.put("END"); // simple sentinel
    });

    // Consumers
    for (int c = 0; c < 3; c++) {
      pool.submit(() -> {
        try {
          for (;;) {
            String e = queue.take();      // blocks if empty
            if ("END".equals(e)) { queue.put("END"); break; } // propagate sentinel
            index(e);
          }
        } catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
      });
    }

    pool.shutdown();
  }
```

```
    static void index(String e) { /* index to search store */ }
}
```

Reasoning to interviewer:

- Bounded queues create natural backpressure—no need for custom flags or sleeps.
- Blocking operations handle coordination; no risk of missed signals or spurious wakeups.
- A sentinel is a simple, safe shutdown protocol for fan-out consumers.

Future scope:

- Replace sentinel with ExecutorService coordinated shutdown or poison pills per consumer.
- Prefer LinkedBlockingQueue for unbounded (with caution) or SynchronousQueue for direct handoff.
- Add metrics on queue depth to autoscale consumers.

Scenario 49: Dynamic team size across phases (CyclicBarrier won't fit)

A data pipeline has phases, but the number of workers changes between phases (some tasks finish early or new ones join). You still need all active workers to rendezvous before moving on.

Answer:

I'll use Phaser. Unlike CyclicBarrier, workers can register/deregister between phases. Each participant calls arriveAndAwaitAdvance() to wait for others in the current phase.

Code:

```
import java.util.concurrent.Phaser;

class Worker implements Runnable {
   private final Phaser phaser;
   private final int myPhases;

   Worker(Phaser phaser, int myPhases) { this.phaser = phaser; this.myPhases =
myPhases; }

   @Override public void run() {
      phaser.register();
      try {
         for (int p = 0; p < myPhases; p++) {
            doWork(p);
            phaser.arriveAndAwaitAdvance(); // rendezvous
         }
      } finally {
         phaser.arriveAndDeregister();
      }
   }

   void doWork(int phase) {
      try { Thread.sleep(200 + (long)(Math.random() * 300)); } catch
(InterruptedException e) { Thread.currentThread().interrupt(); }
      System.out.println(Thread.currentThread().getName() + " completed phase " +
phase);
   }
}

public class PhaserDemo {
   public static void main(String[] args) {
      Phaser phaser = new Phaser(1); // main as a party to prevent premature
termination
```

```
        new Thread(new Worker(phaser, 3), "A").start();
        new Thread(new Worker(phaser, 2), "B").start(); // B leaves earlier
        new Thread(new Worker(phaser, 3), "C").start();

        // main participates until last phase completes
        for (int p = 0; p < 3; p++) phaser.arriveAndAwaitAdvance();
        phaser.arriveAndDeregister();
    }
}
```

Reasoning to interviewer:

- Phaser supports dynamic registration/deregistration, which CyclicBarrier doesn't.
- The "main" party prevents the phaser from terminating before we're done coordinating.
- Deregistration ensures we don't wait for workers that have legitimately finished.

Future scope:

- Use onAdvance hook (subclass Phaser) to log or abort on timeouts.
- Combine with per-phase time budgets and cancellations.
- If phases have distinct thread pools, coordinate pool boundaries via the phaser.

Scenario 50: Timebox and cancel an async pipeline (CompletableFuture)

You fan-out to three services and then enrich the result. If any branch hangs, you want a per-branch timeout, and if enrichment is cancelled, you want the whole chain to stop quickly.

Answer:

I'll compose with CompletableFuture and apply orTimeout(...)/completeOnTimeout(...) per branch. I'll propagate cancellation by cancelling the combined future, and each stage will check interruption/cancellation by not blocking on get() inside stages.

Code:

```
import java.util.concurrent.*;

public class CFTimeoutCancelDemo {
  private static final ExecutorService io = Executors.newFixedThreadPool(4);

  static CompletableFuture<String> slow(String name, long ms) {
    return CompletableFuture.supplyAsync(() -> {
      try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
      return name;
    }, io);
  }

  public static void main(String[] args) {
    CompletableFuture<String> fA = slow("A", 800).orTimeout(500,
TimeUnit.MILLISECONDS)    // will timeout
      .exceptionally(ex -> "A:default");
    CompletableFuture<String> fB = slow("B", 200);
    CompletableFuture<String> fC = slow("C", 300);

    CompletableFuture<String> combined =
      fA.thenCombine(fB, (a, b) -> a + "+" + b)
       .thenCombine(fC, (ab, c) -> ab + "+" + c);

    // Example: cancel whole pipeline based on a condition
    // combined.cancel(true);

    try {
      System.out.println("Result: " + combined.join());
    } finally {
      io.shutdown();
    }
```

```
    }
}
```

Reasoning to interviewer:

- orTimeout completes a future exceptionally if it doesn't finish in time; completeOnTimeout can return a default instead.
- I handle timeouts locally (map to default) so the aggregator still succeeds.
- Avoid blocking get() inside async stages—compose futures instead so cancellation/timeouts propagate naturally.

Future scope:

- Add retries with exponential backoff for transient branches.
- Use anyOf to race equivalent mirrors and take the first winner.
- With virtual threads, you can also write blocking code and rely on cheap cancellation, while keeping composition semantics.

Scenario 51: Parallel streams corrupt output due to shared mutable state

You convert a large list of items to a map using parallelStream(), but occasionally see missing or duplicated keys because you appended to a shared HashMap inside forEach.

Answer:

In parallel streams, avoid mutating non-thread-safe shared state. Use built-in concurrent collectors like toConcurrentMap (or a proper Collector) so accumulation is thread-safe and associative.

Code:

```
import java.util.*;
import java.util.concurrent.ConcurrentMap;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class ParallelStreamSafeCollect {
  public static void main(String[] args) {
    List<Integer> ids = IntStream.rangeClosed(1, 10000).boxed().toList();

    // WRONG: shared HashMap with forEach in parallel
    // Map<Integer, String> bad = new HashMap<>();
    // ids.parallelStream().forEach(i -> bad.put(i, "V" + i)); // data races!

    // RIGHT: use concurrent collector
    ConcurrentMap<Integer, String> good =
      ids.parallelStream()
        .collect(Collectors.toConcurrentMap(i -> i, i -> "V" + i));

    System.out.println("size=" + good.size()); // 10000
  }
}
```

Reasoning to interviewer:

- Parallel pipelines require associative, thread-safe accumulation; mutating a shared HashMap breaks that contract.
- toConcurrentMap uses a concurrent container and merges safely across threads.

• If you must use a custom collector, ensure it’s CONCURRENT and UNORDERED when appropriate.

Future scope:

• For ordered results, collect then sort, or use forEachOrdered (with potential performance cost).

• If the operation is I/O-bound or includes blocking, prefer CompletableFuture or virtual threads over parallel streams.

• Benchmark: for small datasets, sequential streams can outperform due to parallel overhead.

Scenario 52: Threads must finish initialization before main continues

You spin up 5 worker threads to preload caches and initialize connections. The main thread should only start serving requests after all workers report “ready”.

Answer:

I’ll use a CountDownLatch initialized with the worker count. Each worker countDown()s when done, and the main thread calls await(). This makes startup deterministic and avoids polling or sleeps.

Code:

```
import java.util.concurrent.CountDownLatch;

class InitWorker implements Runnable {
   private final CountDownLatch latch;
   private final String name;

   InitWorker(String name, CountDownLatch latch) { this.name = name; this.latch =
latch; }

   @Override public void run() {
      try {
         System.out.println(name + " initializing...");
         Thread.sleep(300 + (int)(Math.random()*200)); // simulate work
         System.out.println(name + " ready");
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } finally {
         latch.countDown();
      }
   }
}

public class StartupLatchDemo {
   public static void main(String[] args) throws InterruptedException {
      CountDownLatch latch = new CountDownLatch(5);
      for (int i=1;i<=5;i++) new Thread(new InitWorker("Worker-"+i, latch)).start();

      latch.await(); // block until all ready
      System.out.println("All workers ready, starting server...");
   }
}
```

Reasoning to interviewer:

- CountDownLatch is a one-shot gate: once count hits zero, all await()ers proceed.
- Better than Thread.sleep() because it adapts to actual work time.
- Always decrement in finally to avoid hanging the system if a worker crashes.

Future scope:

- For repeated barriers, use CyclicBarrier or Phaser.
- Add timeout to await() to avoid indefinite block on stuck workers.
- Wrap init logic in ExecutorService with invokeAll if you need to propagate results or exceptions.

Scenario 53: Need both parallel and sequential phases in a pipeline

You have 10 ETL tasks. They can all extract in parallel, but then must wait before a single sequential consolidation runs. After that, all can transform in parallel.

Answer:

I'll use CyclicBarrier for parallel phases and coordinate the sequential phase via a barrier action. This ensures everyone stops at the end of extract, runs consolidation once, then continues.

Code:

```
import java.util.concurrent.*;

class ETLWorker implements Runnable {
   private final CyclicBarrier barrier;
   private final int id;

   ETLWorker(int id, CyclicBarrier barrier) { this.id=id; this.barrier=barrier; }

   @Override public void run() {
      try {
         extract();
         barrier.await();   // sync after extract
         transform();
         barrier.await();   // sync after transform
         load();
      } catch (Exception e) {
         Thread.currentThread().interrupt();
      }
   }

   void extract(){ System.out.println("T"+id+" extracting"); }
   void transform(){ System.out.println("T"+id+" transforming"); }
   void load(){ System.out.println("T"+id+" loading"); }
}

public class ETLPipeline {
   public static void main(String[] args) {
      int workers=4;
      CyclicBarrier barrier = new CyclicBarrier(workers,
         () -> System.out.println("-- barrier reached, consolidation step --"));

      for (int i=1;i<=workers;i++) new Thread(new ETLWorker(i, barrier)).start();
   }
}
```

Reasoning to interviewer:

- CyclicBarrier fits multi-phase pipelines; its action runs once per phase.
- Each thread does work, then blocks until all others arrive.
- Compared to CountDownLatch, it resets automatically and is reusable.

Future scope:

- For dynamic number of workers, switch to Phaser.
- Add timeouts in await() to prevent one stuck worker blocking all.
- Combine with thread pools for structured task management instead of raw threads.

Scenario 54: Cancel long-running tasks if they exceed a deadline

You run a report generator. Some reports take too long and must be aborted after 5 seconds.

Answer:

I'll submit each task to ExecutorService, hold its Future, and call get(timeout, unit). If it times out, I'll cancel(true) to interrupt the worker.

Code:

```
import java.util.concurrent.*;

class Report implements Callable<String> {
   private final int id;
   Report(int id){this.id=id;}
   @Override public String call() throws Exception {
      Thread.sleep(7000); // simulate slow
      return "Report " + id;
   }
}

public class TimeoutCancelDemo {
   public static void main(String[] args) {
      ExecutorService pool = Executors.newCachedThreadPool();
      Future<String> f = pool.submit(new Report(42));
      try {
         String res = f.get(5, TimeUnit.SECONDS);
         System.out.println("Got: " + res);
      } catch (TimeoutException e) {
         System.out.println("Report took too long, cancelling...");
         f.cancel(true);
      } catch (Exception e) {
         e.printStackTrace();
      } finally {
         pool.shutdown();
      }
   }
}
```

Reasoning to interviewer:

- Future.get(timeout) throws TimeoutException—perfect for deadlines.
- cancel(true) sets interrupt flag; the task must cooperate (e.g., by checking Thread.interrupted() or catching InterruptedException).
- Timeout protects system throughput and avoids unbounded waits.

Future scope:

- Use CompletableFuture.orTimeout() in Java 9+ for cleaner chaining.
- Wrap blocking I/O in libraries that respect interrupts.
- Log timeouts for metrics and possibly retry with backoff.

Scenario 55: Parallel stream runs slower than sequential on small data

You squared a list of 100 ints with parallelStream(), but it runs slower than a simple loop.

Answer:

Parallel streams split work into subtasks via ForkJoinPool. Overhead of splitting and coordination can outweigh benefits on small datasets. For small lists, sequential is better; parallel helps only when:

1. data is large,
2. tasks are CPU-intensive,
3. no shared mutable state.

Code:

```
import java.util.*;
import java.util.stream.*;

public class ParallelPerfDemo {
   public static void main(String[] args) {
      List<Integer> nums = IntStream.rangeClosed(1, 100).boxed().toList();

      long t1 = System.nanoTime();
      nums.stream().map(n->n*n).toList();
      long t2 = System.nanoTime();

      nums.parallelStream().map(n->n*n).toList();
      long t3 = System.nanoTime();

      System.out.printf("Sequential=%.2fms, Parallel=%.2fms%n",
         (t2-t1)/1e6, (t3-t2)/1e6);
   }
}
```

Reasoning to interviewer:

- ForkJoinPool overhead ≈ splitting + task scheduling + join.
- With only 100 elements, cost > work. With millions, parallel wins.
- Don’t blindly switch to parallel; measure per workload.

Future scope:

- Use thresholds (size, CPU intensity) to pick sequential vs parallel.
- For I/O-bound work, prefer CompletableFuture or virtual threads.
- Profile using JMH to quantify actual gains.

Scenario 56: Multiple readers, single writer with high read frequency

You're building a stock price dashboard where dozens of threads read the latest price frequently, while only one thread updates it occasionally. You want high performance for reads without blocking each other.

Answer:

I'd use ReadWriteLock or better, StampedLock. With ReadWriteLock, multiple readers can proceed in parallel but writers are exclusive. With StampedLock, I can even attempt optimistic reads, which don't block unless a write happens.

Code:

```
import java.util.concurrent.locks.StampedLock;

class PriceBoard {
   private final StampedLock lock = new StampedLock();
   private double price = 100.0;

   public void update(double newPrice) {
      long stamp = lock.writeLock();
      try {
         price = newPrice;
      } finally {
         lock.unlockWrite(stamp);
      }
   }

   public double read() {
      long stamp = lock.tryOptimisticRead();
      double val = price;
      if (!lock.validate(stamp)) {  // fallback if a write happened
         stamp = lock.readLock();
         try { val = price; }
         finally { lock.unlockRead(stamp); }
      }
      return val;
   }
}
```

Reasoning to interviewer:

- Reads dominate, so optimistic reads avoid unnecessary locking.
- validate() ensures consistency if a write interfered.
- Writers still get exclusive access when updating.

Future scope:

- If updates increase, switch back to ReentrantReadWriteLock for fairness.
- Add versioning to detect stale reads in distributed scenarios.
- Integrate with caches (e.g., Guava/Redis) for scale-out read-heavy workloads.

Scenario 57: Coordinating different tasks that must run in order

You have 3 steps: load data, process it, and finally save results. Each step must wait for the previous one to complete across multiple threads.

Answer:

I'd use a CountDownLatch chain or CompletableFuture chaining. With latch chaining, each stage waits for the previous latch to reach zero before proceeding.

Code:

```
import java.util.concurrent.CountDownLatch;

public class OrderedTasksDemo {
  public static void main(String[] args) throws InterruptedException {
    CountDownLatch loadDone = new CountDownLatch(1);
    CountDownLatch processDone = new CountDownLatch(1);

    Thread loader = new Thread(() -> {
      System.out.println("Loading data...");
      loadDone.countDown();
    });

    Thread processor = new Thread(() -> {
      try {
        loadDone.await();
        System.out.println("Processing data...");
        processDone.countDown();
      } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });

    Thread saver = new Thread(() -> {
      try {
        processDone.await();
        System.out.println("Saving results...");
      } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    });

    loader.start(); processor.start(); saver.start();
  }
}
```

Reasoning to interviewer:

- Each stage gates on the previous latch, ensuring order.
- This avoids busy waiting or sleeps.
- For complex workflows, CompletableFuture.thenRun() or orchestration frameworks may be better.

Future scope:

- For DAG-style dependencies, use CompletableFuture chaining.
- Add error handling and early termination if one stage fails.
- Move to async/reactive frameworks for large dependency graphs.

Scenario 58: Detect and recover from livelock

Two worker threads politely keep yielding control when they detect contention, but they never make progress (classic livelock).

Answer:

This is livelock: threads are active but not progressing. To fix, add a random back-off strategy so they don't both retry at the same time.

Code:

```
class LivelockFixed {
  private volatile boolean resourceInUse = false;

  public void useResource(String worker) {
    while (true) {
      if (!resourceInUse) {
        resourceInUse = true;
        System.out.println(worker + " acquired resource");
        try { Thread.sleep(200); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
        resourceInUse = false;
        System.out.println(worker + " released resource");
        break;
      } else {
        try {
          // backoff prevents livelock
          Thread.sleep((int)(Math.random()*100));
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
      }
    }
  }
}
```

Reasoning to interviewer:

- Livelock isn't a dead halt (like deadlock), but no useful work gets done.
- Random or exponential backoff breaks the symmetry.
- Real systems (e.g., networking, DB retries) use this to avoid livelock storms.

Future scope:

- Use Lock.tryLock(timeout) instead of manual busy loops.
- Add metrics for retry count to detect potential livelocks.
- Apply fairness policies if many threads contend repeatedly.

Scenario 59: Cancel tasks cleanly in ExecutorService

You schedule tasks that may hang (e.g., external API calls). You want to cancel them if a signal is received (like user logout).

Answer:

I'd hold on to Future<?> objects returned by submit() and call cancel(true). Each task should check Thread.interrupted() periodically to exit gracefully.

Code:

```
import java.util.concurrent.*;

class CancellableTask implements Runnable {
   public void run() {
      try {
         for (int i=0;i<10;i++) {
            if (Thread.interrupted()) {
               System.out.println("Task interrupted, cleaning up...");
               return;
            }
            Thread.sleep(500);
            System.out.println("Step " + i);
         }
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
         System.out.println("Interrupted during sleep, exiting...");
      }
   }
}

public class CancelTasksDemo {
   public static void main(String[] args) throws InterruptedException {
      ExecutorService pool = Executors.newSingleThreadExecutor();
      Future<?> f = pool.submit(new CancellableTask());
      Thread.sleep(1200);
      f.cancel(true); // cancel after 1.2s
      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- cancel(true) sets interrupt flag, which must be checked in the task.
- Cooperative cancellation avoids resource leaks and half-updates.
- Threads ignoring interrupts will appear “uncancellable”.

Future scope:

- Use CompletableFuture.cancel() for async pipelines.
- Add cleanup hooks (closing sockets, rolling back DB).
- Centralize cancellation via ExecutorService.shutdownNow().

Scenario 60: Parallel aggregation of large datasets

You need to compute total sales from a dataset of millions of records. Single-threaded loops are too slow.

Answer:

I'd use Java 8 parallel streams or ForkJoinPool to split the work across cores. With parallel streams, the workload is automatically partitioned and reduced.

Code:

```
import java.util.*;
import java.util.stream.*;

public class SalesAggregator {
   public static void main(String[] args) {
      List<Integer> sales = IntStream.range(1, 1_000_000).boxed().toList();

      long total = sales.parallelStream()
                  .mapToLong(Integer::longValue)
                  .sum();

      System.out.println("Total sales = " + total);
   }
}
```

Reasoning to interviewer:

- Parallel streams leverage ForkJoinPool with work-stealing.
- Operations must be associative and stateless for correctness.
- Aggregation scales well when dataset is large and CPU-bound.

Future scope:

- Use Collectors.groupingByConcurrent for grouped aggregations.
- For I/O-heavy tasks, switch to async APIs instead of parallel streams.
- Tune ForkJoinPool parallelism if default doesn't suit workload.

Scenario 61: Throttling API calls with limited concurrency

You're writing a service that calls an external API, but the API provider allows only 3 concurrent calls at a time. Extra calls should wait.

Answer:

I'd use a Semaphore with 3 permits. Each API call thread acquires a permit before sending the request and releases it after completion.

Code:

```
import java.util.concurrent.Semaphore;

class ApiClient {
   private final Semaphore semaphore = new Semaphore(3); // max 3 concurrent calls

   public void callApi(String name) {
      try {
         semaphore.acquire();
         System.out.println(name + " started API call");
         Thread.sleep(1000); // simulate call
         System.out.println(name + " finished API call");
      } catch (InterruptedException e) {
         Thread.currentThread().interrupt();
      } finally {
         semaphore.release();
      }
   }
}

public class ApiThrottleDemo {
   public static void main(String[] args) {
      ApiClient client = new ApiClient();
      for (int i = 1; i <= 6; i++) {
         String name = "Task-" + i;
         new Thread(() -> client.callApi(name)).start();
      }
   }
}
```

Reasoning to interviewer:

- Semaphore models the number of available “slots” for resource access.
- Threads block automatically if permits are unavailable.
- This prevents overload without hand-coded queue management.

Future scope:

- Use tryAcquire(timeout) for fail-fast instead of indefinite waiting.
- Track rejected/blocked calls for monitoring.
- Integrate with rate-limiters (like Guava’s RateLimiter) for QPS control.

Scenario 62: Coordinating variable participants across phases

You're simulating a game tournament where players can join or leave between rounds. At the end of each round, all current players must sync before moving to the next.

Answer:

I'd use Phaser, which supports dynamic registration/deregistration unlike CyclicBarrier. Each player registers, plays, and waits at arriveAndAwaitAdvance().

Code:

```
import java.util.concurrent.Phaser;

class Player implements Runnable {
   private final Phaser phaser;
   private final int rounds;
   private final String name;

   Player(String name, Phaser phaser, int rounds) {
      this.name = name; this.phaser = phaser; this.rounds = rounds;
      phaser.register();
   }
   @Override public void run() {
      try {
         for (int i = 1; i <= rounds; i++) {
            System.out.println(name + " finished round " + i);
            phaser.arriveAndAwaitAdvance();
         }
      } finally {
         phaser.arriveAndDeregister();
      }
   }
}

public class PhaserDemo {
   public static void main(String[] args) {
      Phaser phaser = new Phaser(1); // register main
      new Thread(new Player("Alice", phaser, 3)).start();
      new Thread(new Player("Bob", phaser, 2)).start();

      for (int i = 1; i <= 3; i++) {
         phaser.arriveAndAwaitAdvance();
         System.out.println("Round " + i + " completed.");
      }
      phaser.arriveAndDeregister();
   }
```

```
}
```

Reasoning to interviewer:

- Phaser is more flexible than CyclicBarrier for dynamic participation.
- Deregistration prevents waiting on threads that left early.
- Useful for simulations, iterative algorithms, or variable teams.

Future scope:

- Override onAdvance to add custom phase logic.
- Add timeouts to detect stuck participants.
- For hierarchical workflows, use multiple phasers.

Scenario 63: Task dependencies with CompletableFuture

In an order processing system, you must:

1. Validate the order.
2. Charge payment only if validation succeeds.
3. Send confirmation email after payment.

Answer:

I'd chain CompletableFutures using thenCompose() for dependent tasks and thenRun() for actions without results.

Code:

```
import java.util.concurrent.*;

public class OrderPipelineDemo {
  public static void main(String[] args) {
    ExecutorService executor = Executors.newFixedThreadPool(3);

    CompletableFuture<Void> pipeline =
      CompletableFuture.supplyAsync(() -> validateOrder("ORD123"), executor)
      .thenCompose(valid -> chargePayment(valid, executor))
      .thenAccept(paymentId -> sendConfirmation(paymentId));

    pipeline.join();
    executor.shutdown();
  }

  static boolean validateOrder(String id) {
    System.out.println("Validating " + id);
    return true;
  }
  static CompletableFuture<String> chargePayment(boolean valid, Executor
executor) {
    return CompletableFuture.supplyAsync(() -> {
      if (!valid) throw new RuntimeException("Invalid order");
      System.out.println("Charging payment...");
      return "TXN-999";
    }, executor);
  }
  static void sendConfirmation(String txnId) {
    System.out.println("Sending confirmation for " + txnId);
  }
}
```

Reasoning to interviewer:

- thenCompose flattens dependent async tasks.
- thenAccept consumes a result without producing a new one.
- Clear pipeline, avoids blocking, keeps error handling async.

Future scope:

- Add .exceptionally() or .handle() for fallback logic.
- Use allOf() when processing multiple parallel validations.
- Combine with timeouts for resilience.

Scenario 64: Handling starvation in lock-heavy system

A high-priority thread keeps grabbing a lock, while other threads rarely progress.

Answer:

This is starvation. I'd use a fair ReentrantLock(true) to ensure FIFO lock acquisition.

Code:

```
import java.util.concurrent.locks.ReentrantLock;

class SharedCounter {
   private final ReentrantLock lock = new ReentrantLock(true); // fair lock
   private int counter = 0;

   public void increment(String name) {
      lock.lock();
      try {
         counter++;
         System.out.println(name + " incremented to " + counter);
      } finally {
         lock.unlock();
      }
   }
}

public class FairLockDemo {
   public static void main(String[] args) {
      SharedCounter counter = new SharedCounter();
      for (int i = 1; i <= 5; i++) {
         String t = "T" + i;
         new Thread(() -> counter.increment(t)).start();
      }
   }
}
```

Reasoning to interviewer:

- Fair locks grant access roughly in order of request, avoiding starvation.
- Comes with throughput tradeoff compared to non-fair locks.
- Starvation often occurs when some threads hog resources repeatedly.

Future scope:

- Use lock sharding or finer-grained locks to reduce contention.
- For read-heavy workloads, use ReadWriteLock.
- Monitor lock wait times in production to detect hidden starvation.

Scenario 65: Run thousands of tasks efficiently

You have 10,000 short tasks (like sending notifications). Creating a thread per task is too costly.

Answer:

I'd use a fixed-size thread pool (ExecutorService) to reuse threads and manage queued tasks.

Code:

```
import java.util.concurrent.*;

class NotifyTask implements Runnable {
   private final int id;
   NotifyTask(int id) { this.id = id; }
   public void run() {
      System.out.println("Sending notification " + id + " by " +
Thread.currentThread().getName());
   }
}

public class ThreadPoolDemo {
   public static void main(String[] args) {
      ExecutorService pool = Executors.newFixedThreadPool(10);

      for (int i = 1; i <= 20; i++) {
         pool.submit(new NotifyTask(i));
      }

      pool.shutdown();
   }
}
```

Reasoning to interviewer:

- Thread pools amortize thread creation cost.
- A bounded pool prevents resource exhaustion.
- Tasks are queued when all threads are busy.

Future scope:

- Use newCachedThreadPool for bursty workloads.
- For scheduled jobs, use ScheduledExecutorService.
- Consider virtual threads (Java 21+) for massive concurrency.

Scenario 66: Divide-and-conquer over a big array (ForkJoinPool)

You need to sum a massive array (or apply a CPU-heavy transform) faster than a single thread can manage. The work splits naturally.

Answer:

I'll use ForkJoinPool with a RecursiveTask<Long>. Each task splits the array until chunks are small, computes locally, and combines results. This keeps cores busy with work-stealing and minimizes coordination overhead.

Code:

```
import java.util.concurrent.*;

class SumTask extends RecursiveTask<Long> {
   private static final int THRESHOLD = 10_000;
   private final long[] arr;
   private final int lo, hi;

   SumTask(long[] arr, int lo, int hi) { this.arr = arr; this.lo = lo; this.hi = hi; }

   @Override protected Long compute() {
      int len = hi - lo;
      if (len <= THRESHOLD) {
         long sum = 0;
         for (int i = lo; i < hi; i++) sum += arr[i];
         return sum;
      }
      int mid = lo + len / 2;
      SumTask left = new SumTask(arr, lo, mid);
      SumTask right = new SumTask(arr, mid, hi);
      left.fork();
      long r = right.compute();     // compute one half directly
      long l = left.join();         // then join the other half
      return l + r;
   }
}

public class ForkJoinSumDemo {
   public static void main(String[] args) {
      long[] data = new long[5_000_000];
      for (int i = 0; i < data.length; i++) data[i] = i % 10;

      ForkJoinPool pool = new ForkJoinPool(); // or custom parallelism
      long total = pool.invoke(new SumTask(data, 0, data.length));
      System.out.println("Total = " + total);
```

```
        pool.shutdown();
    }
}
```

Reasoning to interviewer:

- ForkJoinPool uses work-stealing, which keeps threads busy and reduces contention.
- Splitting until a threshold balances parallel overhead vs. work size.
- Calling compute() on one branch and fork()/join() on the other improves locality.

Future scope:

- Tune THRESHOLD with JMH.
- Use RecursiveAction for in-place transforms.
- Provide a custom pool when the common pool is busy with other tasks.

Scenario 67: Process results as soon as they finish (ExecutorCompletionService)

You launch multiple downstream calls (pricing, shipping, tax). Each completes at different times; you want to stream results immediately instead of waiting for all.

Answer:

I'll wrap a fixed thread pool with ExecutorCompletionService. Submit N callables, then pull take() in a loop to get futures in completion order. Handle exceptions per-future to avoid stalling the loop.

Code:

```
import java.util.concurrent.*;

class FetchTask implements Callable<String> {
   private final String name; private final long delayMs;
   FetchTask(String name, long delayMs) { this.name = name; this.delayMs = delayMs;
}
   @Override public String call() throws Exception {
      Thread.sleep(delayMs);
      if (Math.random() < 0.2) throw new RuntimeException(name + " failed");
      return name + " OK";
   }
}

public class CompletionServiceDemo {
   public static void main(String[] args) throws InterruptedException {
      ExecutorService pool = Executors.newFixedThreadPool(3);
      ExecutorCompletionService<String> ecs = new
ExecutorCompletionService<>(pool);

      ecs.submit(new FetchTask("pricing", 800));
      ecs.submit(new FetchTask("shipping", 300));
      ecs.submit(new FetchTask("tax", 600));

      for (int i = 0; i < 3; i++) {
         Future<String> f = ecs.take();
         try {
            System.out.println("-> " + f.get());
         } catch (ExecutionException e) {
            System.out.println("-> error: " + e.getCause().getMessage());
         }
      }
      pool.shutdown();
   }
```

```
}
```

Reasoning to interviewer:

- Completion queue decouples submission order from completion order—great for responsiveness.
- Per-future try/catch prevents one failure from killing the whole loop.
- A bounded pool keeps concurrency under control.

Future scope:

- Use poll(timeout) to surface stragglers.
- Attach correlation IDs for partial rendering.
- Combine with a circuit breaker when failures spike.

Scenario 68: Robust async pipeline with fallbacks (CompletableFuture)

You call three services; any one might fail. You want defaults for failures, still aggregate everything, and log what broke—without blocking.

Answer:

I'll compose with CompletableFuture, apply exceptionally/handle on each branch to convert failures into safe defaults, then allOf and aggregate. No get() in the pipeline—only at the edges.

Code:

```
import java.util.concurrent.*;

public class CFErrorHandlingDemo {
  private static final ExecutorService io = Executors.newFixedThreadPool(4);

  static CompletableFuture<String> svc(String name, long ms, boolean fail) {
    return CompletableFuture.supplyAsync(() -> {
      try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
      if (fail) throw new RuntimeException(name + " boom");
      return name + ":data";
    }, io);
  }

  public static void main(String[] args) {
    CompletableFuture<String> a = svc("A", 300, false)
        .exceptionally(ex -> { System.out.println("A failed: " + ex.getMessage()); return
"A:default"; });

    CompletableFuture<String> b = svc("B", 600, true)
        .handle((val, ex) -> ex == null ? val : "B:default");

    CompletableFuture<String> c = svc("C", 200, false)
        .exceptionally(ex -> "C:default");

    String result = CompletableFuture.allOf(a, b, c)
        .thenApply(v -> String.join("|", a.join(), b.join(), c.join()))
        .join();

    System.out.println("Aggregated -> " + result);
    io.shutdown();
  }
}
```

Reasoning to interviewer:

- Handle errors at the leaf futures so the aggregator doesn't explode.
- exceptionally maps failure → fallback; handle inspects both success/failure.
- allOf waits for completion; join() is safe afterward and avoids checked exceptions noise.

Future scope:

- Add per-branch orTimeout and retries with backoff.
- Collect error metrics to tune fallbacks.
- Use anyOf to race mirrors for latency-sensitive branches.

Scenario 69: Double-buffer handoff between two threads (Exchanger)

One thread fills a buffer (producer), the other drains it (consumer). You want zero copying and to swap buffers each cycle.

Answer:

I'll use Exchanger<T> to exchange buffer references. Each thread calls exchange(buffer): the producer hands off a full buffer and receives an empty one; the consumer does the opposite.

Code:

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Exchanger;

public class ExchangerDemo {
  public static void main(String[] args) {
    Exchanger<List<Integer>> ex = new Exchanger<>();
    List<Integer> empty = new ArrayList<>();
    List<Integer> full = new ArrayList<>();
    Thread producer = new Thread(() -> {
      List<Integer> buf = empty;
      try {
        for (int round = 0; round < 5; round++) {
          buf.clear();
          for (int i = 0; i < 5; i++) buf.add(round * 10 + i);
          buf = ex.exchange(buf); // hand off full; get empty back
        }
      } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }, "producer");
    Thread consumer = new Thread(() -> {
      List<Integer> buf = full;
      try {
        for (int round = 0; round < 5; round++) {
          buf = ex.exchange(buf); // receive full; return empty
          System.out.println("Consumed: " + buf);
          buf.clear(); // now empty for next round
        }
      } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }, "consumer");

    producer.start(); consumer.start();
  }
}
```

Reasoning to interviewer:

- Exchanger swaps references atomically—no copying, minimal GC.
- Ideal for two-party pipelines with batch boundaries.
- Both sides synchronize naturally at each exchange.

Future scope:

- Add timeouts to exchange() for robustness.
- For more than two parties, use queues/barriers instead.
- Use object pools for reusable buffers and fewer allocations.

Scenario 70: Graceful shutdown of multi-consumer pipeline (BlockingQueue + poison pill)

You have a bounded queue with many consumers. When the producer finishes, all consumers should exit cleanly—no hanging threads.

Answer:

I'll insert one poison pill per consumer after production ends. Each consumer exits when it dequeues the pill. This avoids races around "is-done" flags.

Code:

```
import java.util.concurrent.*;
import java.util.*;

public class PoisonPillDemo {
  private static final String PILL = "__STOP__";

  public static void main(String[] args) throws InterruptedException {
    int consumers = 3;
    BlockingQueue<String> q = new ArrayBlockingQueue<>(10);
    ExecutorService pool = Executors.newFixedThreadPool(consumers);

    // Consumers
    for (int i = 0; i < consumers; i++) {
      pool.submit(() -> {
        try {
          for (;;) {
            String msg = q.take();
            if (PILL.equals(msg)) break;
            // process
            System.out.println(Thread.currentThread().getName() + " -> " + msg);
          }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
      });
    }

    // Producer
    for (int i = 1; i <= 20; i++) q.put("job-" + i);

    // Stop signal: one pill per consumer
    for (int i = 0; i < consumers; i++) q.put(PILL);

    pool.shutdown();
```

```
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("Pipeline stopped cleanly");
    }
}
```

Reasoning to interviewer:

- A bounded BlockingQueue gives built-in backpressure.
- One poison pill per consumer guarantees all workers see a stop signal.
- No fragile shared flags or timing assumptions.

Future scope:

- Replace pills with ExecutorService shutdown + queue drain if tasks are cancellable.
- Add retries/dead-letter queue for failures.
- Track queue depth for autoscaling consumers.